ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

# BÀI 6. Behavioral analysis - Modeling

# Mô hình hóa hành vi ( ứng xử )

hung.letan@hust.edu.vn

0913030731

ONE LOVE. ONE FUTURE.

# Mục tiêu Buổi học

- Hiểu vai trò của mô hình hóa hành vi trong việc mô tả khía cạnh động của hệ thống.
- Biết cách mô tả vòng đời của một đối tượng phức tạp bằng
  - Sơ đồ Máy trạng thái Hành vi (Behavioral State Machine).
- Nắm vững Sơ đồ Tương tác (Interaction Diagrams) để mô tả sự hợp tác giữa các đối tượng dựa trên
  - Sơ đồ Chuỗi (Sequence Diagram) và
  - Sơ đồ Giao tiếp (Communication Diagram).
    - **Interaction Overview Diagram** (IOD) và Timing Diagram
- Sử dụng Ma trận CRUD để phân tích và xác thực tương tác cấp hệ thống.
  - Hiểu quy trình xác minh và xác thực (verification and validation) các mô hình hành vi

# Giới thiệu

- mô hình chức năng và cấu trúc Functional Model Structural Model.
  - 'bản vẽ kiến trúc tĩnh' của hệ thống
  - Cho biết hệ thống cần làm gì ? và được cấu tạo từ những gì?

- câu hỏi'
  - Các thành phần thay đổi như thế nào theo thời gian?'.
- Đây chính là nội dung cốt lõi của Mô hình hóa Hành vi - behavioral modeling
  - Tạo ra góc nhìn động (Dynamic View) của hệ thống
  - làm cho bản thiết kế (blueprint) trở nên sống động  và có  hồn,

# Behavioral modeling

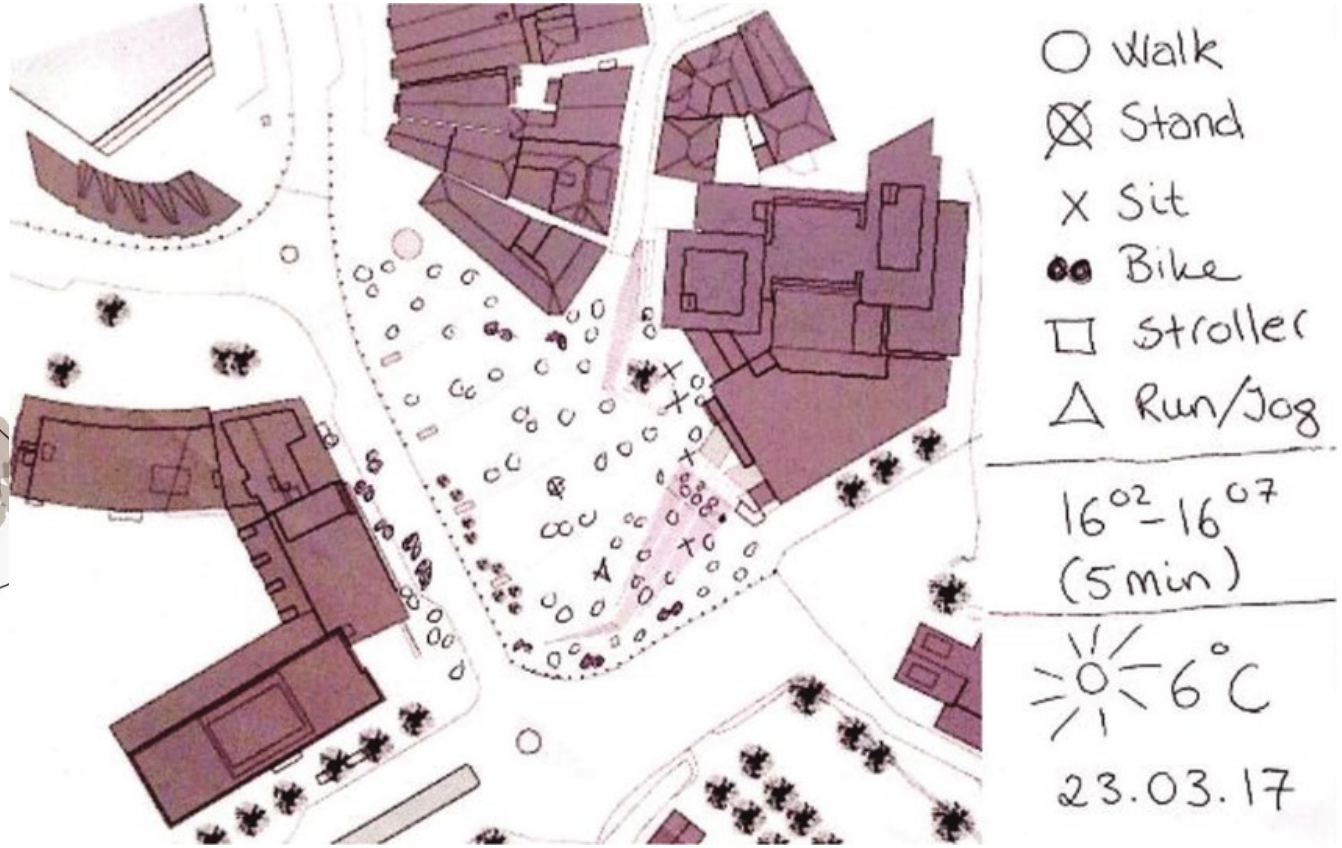
- Mô hình hóa hành vi (OOAD)
  - tập trung vào việc nắm bắt các khía cạnh động của hệ thống bằng cách mô tả các tương tác giữa các đối tượng và luồng điều khiển

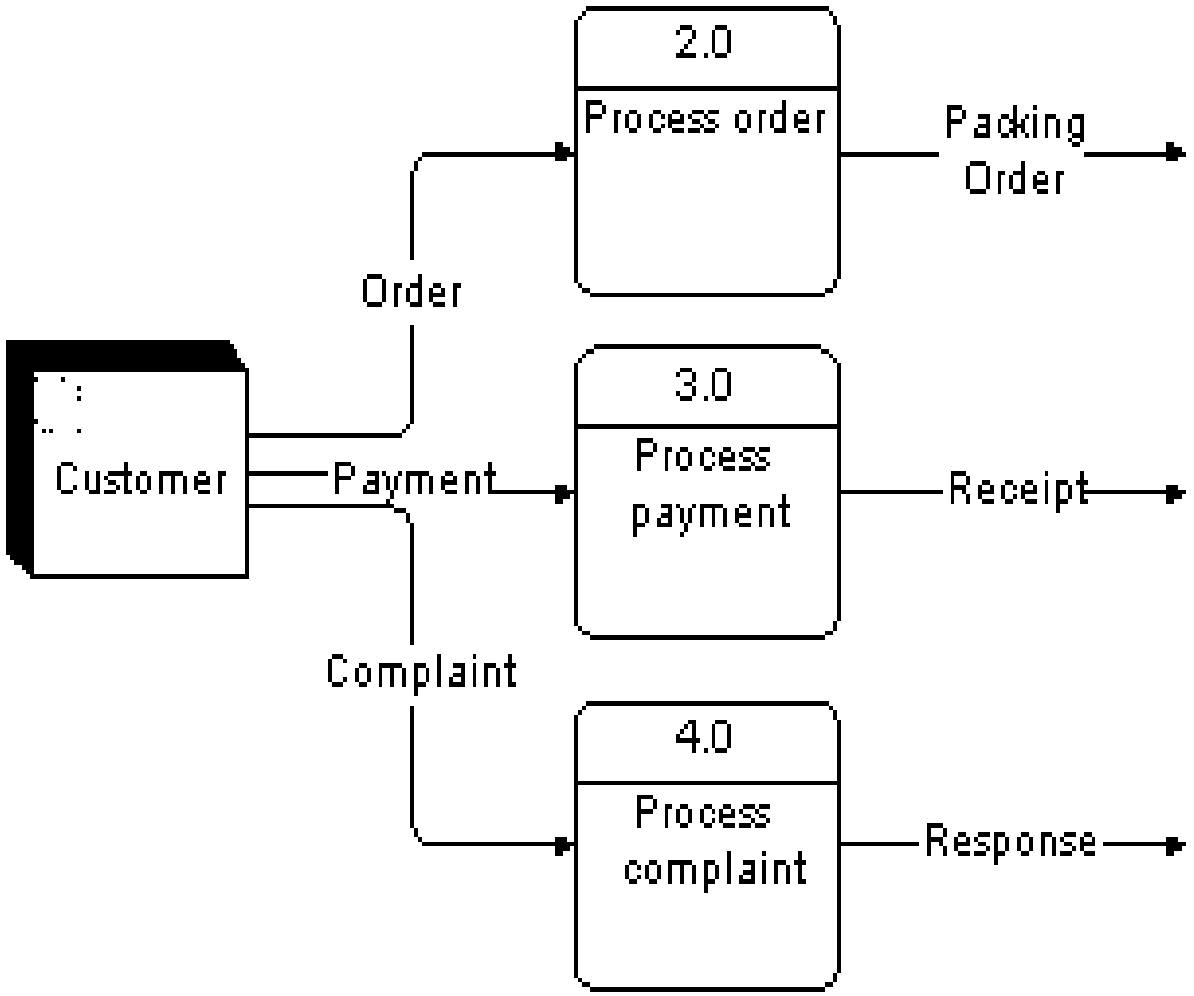
- "Mô hình hóa hành vi không phải là một hoạt động độc lập.
  - Nó là một mắt xích quan trọng, kết nối chặt chẽ giữa Giai đoạn Phân tích và Giai đoạn Thiết kế,
  - giúp biến các yêu cầu thành một bản thiết kế có thể triển khai được.“
  - Quá trình này có tính lặp đi lặp lại và tăng dần (iterative and incremental)

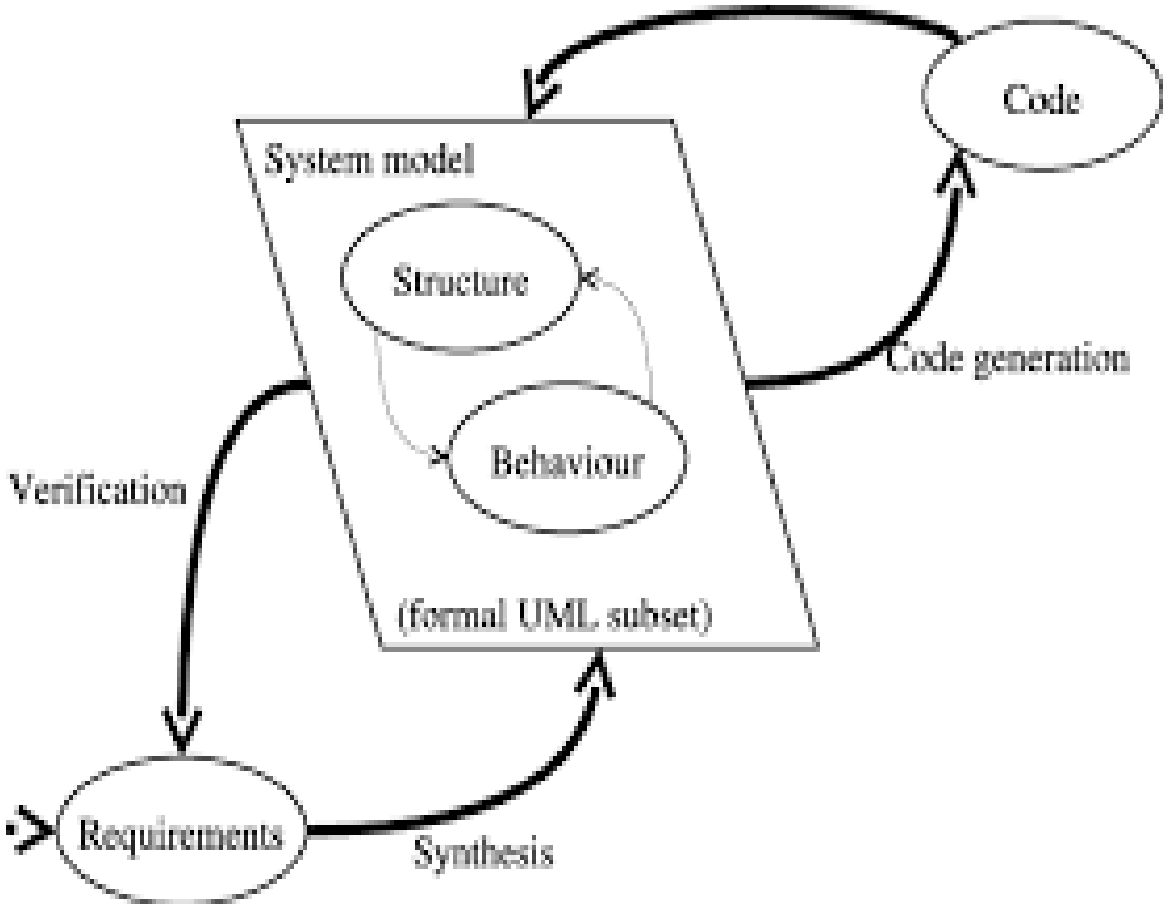
# Đặc điểm:

- Thuộc về Giai đoạn Phân tích (Analysis Phase),
  - thuộc quy trình công việc Phân tích (Analysis workflow).
  - Cung cấp góc nhìn động (dynamic view) hoặc góc nhìn hành vi (behavioral view) về hệ thống.
- Đầu vào:
  - Các mô hình chức năng (Use Case, Activity Diagram) và
  - mô hình cấu trúc (Class Diagram).
- Đầu ra:
  - Các mô hình động (Interaction Diagrams, State Machines)
  - làm tiền đề cho Giai đoạn Thiết kế (Design Phase),
    - đặc biệt là thiết kế lớp và phương thức (Class and Method Design).




Walk
Stand
Sit
Bike
Stroller
Run/Jog
16^02-16^07
(5min)
6°C
23.03.17


2.0
Process order
Packing
Order
Order
Customer
3.0
Process
payment
Payment
Receipt
Complaint
4.0
Process
complaint
Response


Code
System model
Structure
Behaviour
(formal UML subset)
Code generation
Verification
Requirements
Synthesis

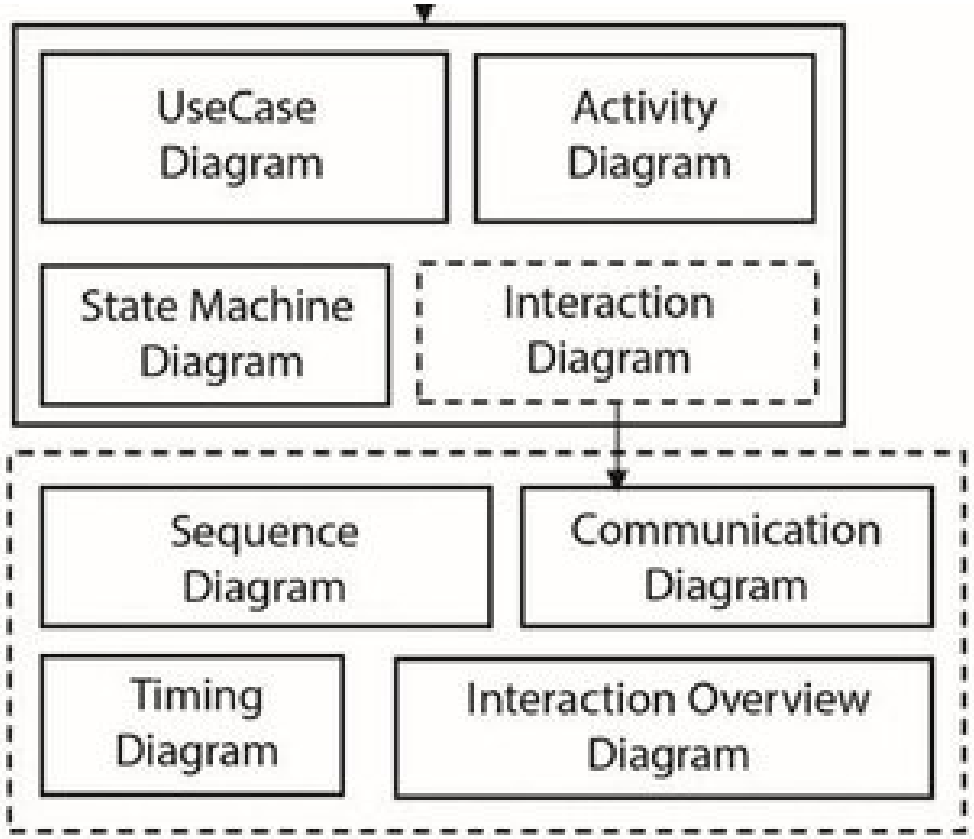
# Khái niệm hành vi

- Chúng ta biết rằng trong suốt vòng đời của mình,
  - một số đối tượng trong hệ thống không hề tĩnh tại;
  - chúng liên tục thay đổi để phản ứng lại với các sự kiện xảy ra.
- Tương tự trong một hệ thống thực tế, các đối tượng không tồn tại một cách biệt lập.
  - chúng là một xã hội thu nhỏ, liên tục hợp tác, trao đổi thông tin để cùng nhau hoàn thành một công việc lớn hơn
- Hành vi
  - Hành vi (hay động thái) là sự hoạt động của các đối tượng nhằm tạo ra các kịch bản
  - Hành vi bao gồm tương tác (trao đổi thông điệp) và ứng xử (phản ứng với các sự kiện)

# Cách tiếp cận

- "Để mô tả hành vi động của hệ thống, chúng ta có hai cách tiếp cận chính:

1. xem xét sự thay đổi trạng thái bên trong một đối tượng duy nhất.“
   - Mô hình trạng thái (State Models):
   - Mô tả vòng đời của một đối tượng, cho thấy các trạng thái khác nhau mà nó trải qua và các sự kiện gây ra sự thay đổi.
2. xem xét sự tương tác giữa nhiều đối tượng với nhau,
   - Mô hình tương tác (Interaction Models):
   - Mô tả sự hợp tác và trao đổi thông điệp giữa các đối tượng để thực hiện một tác vụ.



# State Models
# State Machine Diagram

# Ví dụ, về một đối tượng DonHang (Order)

- Một đơn hàng không chỉ tồn tại đơn thuần.
  - Nó trải qua một vòng đời:
    - từ lúc được tạo, chờ xử lý, chờ thanh toán, được vận chuyển, và cuối cùng là hoàn thành.
- Tương tự, một chiếc điện thoại không phải lúc nào cũng ở trạng thái "nghe-gọi".
  - Nó có thể đang "tắt nguồn", "chờ", "đang sạc", hoặc "đang gọi".
- Để mô hình hóa vòng đời của một đối tượng phức tạp như vậy, chúng ta cần một công cụ đặc biệt.
  - Đó chính là Mô hình Trạng thái (State Models),
  - và công cụ chính để thể hiện nó là Sơ đồ Máy trạng thái Hành vi (Behavioral State Machine)

# Các Thành phần Cốt lõi của Mô hình Trạng thái

- trước khi đi vào chi tiết của Sơ đồ Máy trạng thái, các bạn cần nắm vững bộ ba khái niệm cốt lõi :
  1. Đối tượng tồn tại trong các Trạng thái,
  2. nó Chuyển đổi giữa các trạng thái này
  3. khi một Sự kiện xảy ra.

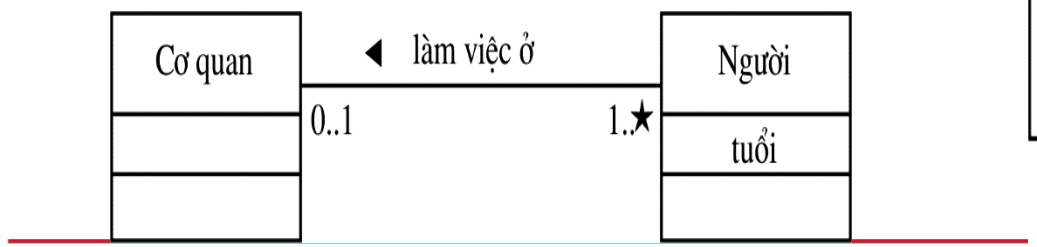
- Việc mô hình hóa những yếu tố này giúp chúng ta nắm bắt được các quy tắc nghiệp vụ phức tạp và vòng đời của những đối tượng quan trọng nhất trong hệ thống.

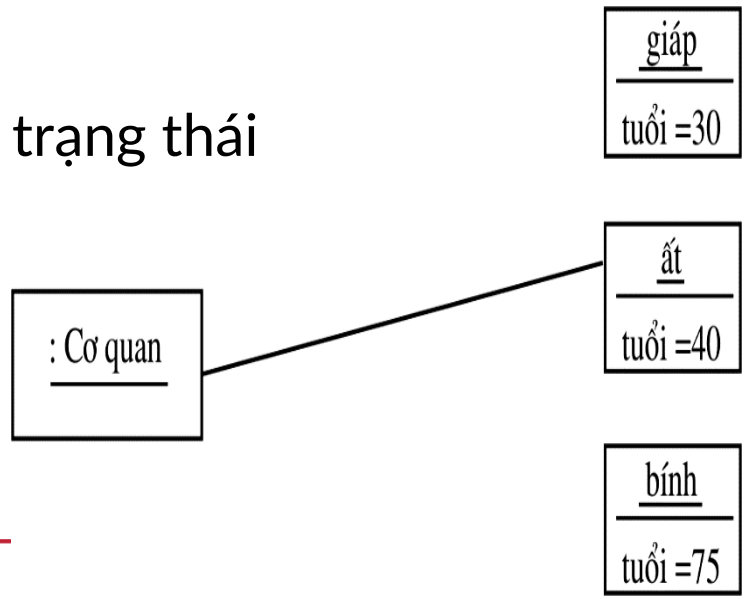
# 1. Trạng thái (State)

- State -Trạng thái
  - là một điều kiện hoặc tình huống trong vòng đời của một đối tượng,
  - tại đó nó thỏa mãn một tiêu chí, thực hiện một hành động, hoặc chờ một sự kiện.
- Trạng thái của đối tượng được xác định bởi
  - giá trị các thuộc tính của nó và
  - mối quan hệ của nó với các đối tượng khác tại một thời điểm cụ thể.
- Một đối tượng sẽ tồn tại trong một trạng thái trong một khoảng thời gian hữu hạn
- Ví dụ,.
  - một đơn hàng đang ở trạng thái Đang chờ xử lý.
  - Một chiếc đèn đang ở trạng thái Bật.
  - Một sinh viên đang ở trạng thái Đã đăng ký.

# Ví dụ: trạng thái

- Với mỗi đối tượng trong lớp Người, thì tuỳ thuộc vào:
  - tuổi của người đó, và
  - có kết nối với một cơ quan nào không,
- ở một trong 3 trạng thái sau:
  - Có việc làm (có kết nối với một cơ quan)
  - Thất nghiệp (tuổi < 60 và không kết nối với cơ quan nào)
  - Nghỉ hưu (tuổi > 60)
- Với biểu đồ trên,
  - thì đối tượng Giáp ở trạng thái "thất nghiệp", đối tượng Ất ở trạng thái "có việc làm", và đối tượng Bính ở trạng thái "nghỉ hưu"



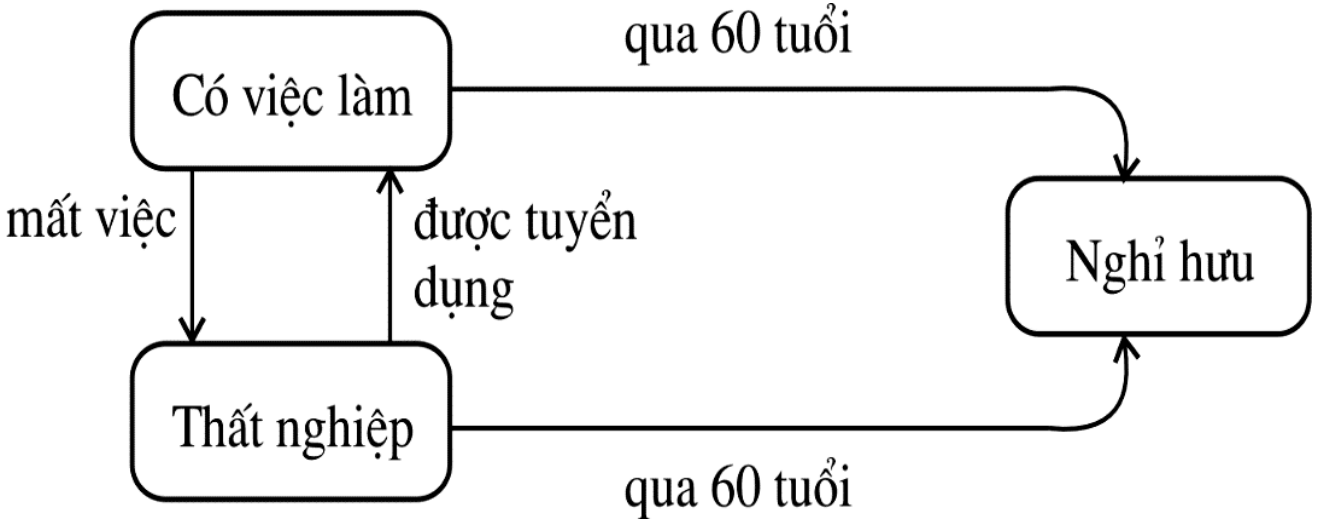
# 2. Sự kiện (Event)

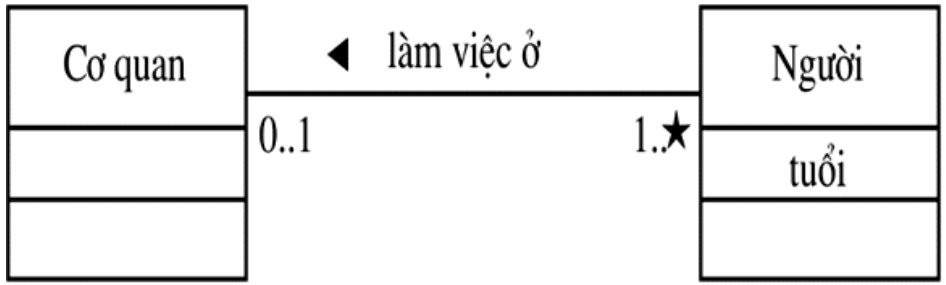
- Sự kiện
  - là một điều đáng chú ý xảy ra tại một thời điểm và không gian nhất định,
  - có khả năng kích hoạt sự thay đổi trạng thái của đối tượng.
- Phân loại:
  - Một tín hiệu (signal) được gửi đi.
  - Một lời gọi phương thức (call).
  - Sự trôi qua của một khoảng thời gian (passage of time).
  - Một sự thay đổi điều kiện (change in state)
- Ví dụ,
  - sự kiện Khách hàng thanh toán xảy ra, làm cho đơn hàng chuyển trạng thái.
  - Sự kiện Công tắc được bật xảy ra, làm chiếc đèn thay đổi.
  - Nếu "trạng thái" là danh từ chỉ tình trạng, thì "sự kiện" chính là động từ gây ra sự thay đổi.

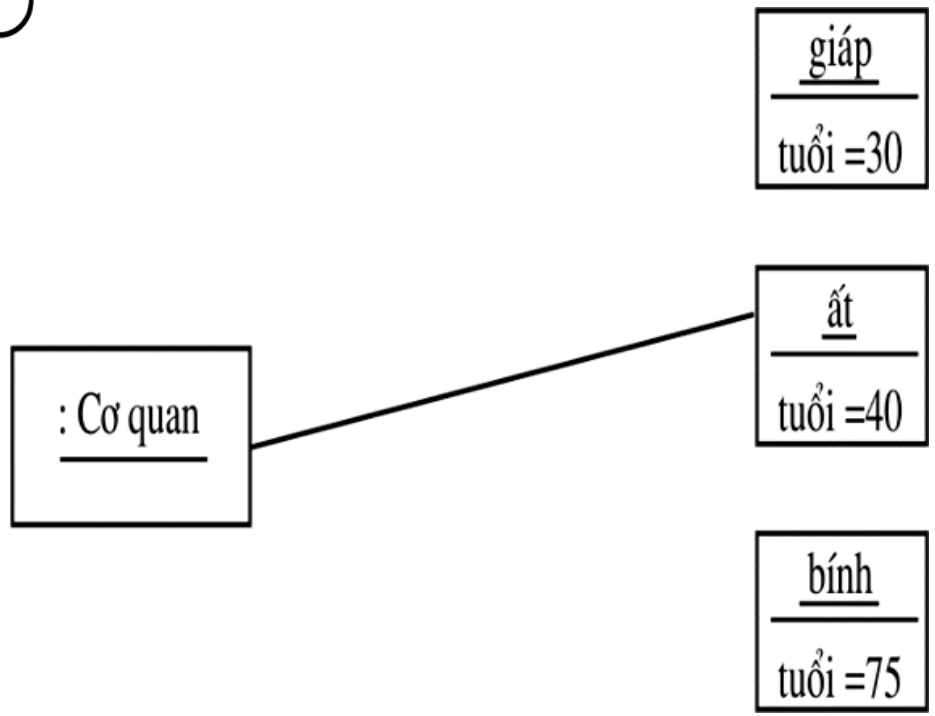
# 3. Chuyển đổi (Transition)

- Transition Chuyển đổi
  - là sự di chuyển của một đối tượng từ một trạng thái (nguồn - origin state) sang một trạng thái khác (đích - destination state).
- Thành phần của một chuyển đổi:
  - Một chuyển đổi thường được ghi nhãn với cú pháp:
  - ten_su_kien (tham_so) [dieu_kien_bao_ve] / bieu_thuc_hanh_dong.
- "Chuyển đổi" là cây cầu nối giữa hai trạng thái, và "sự kiện" chính là thứ mở cánh cổng cho đối tượng đi qua cây cầu đó.

# Vd Các dịch chuyển

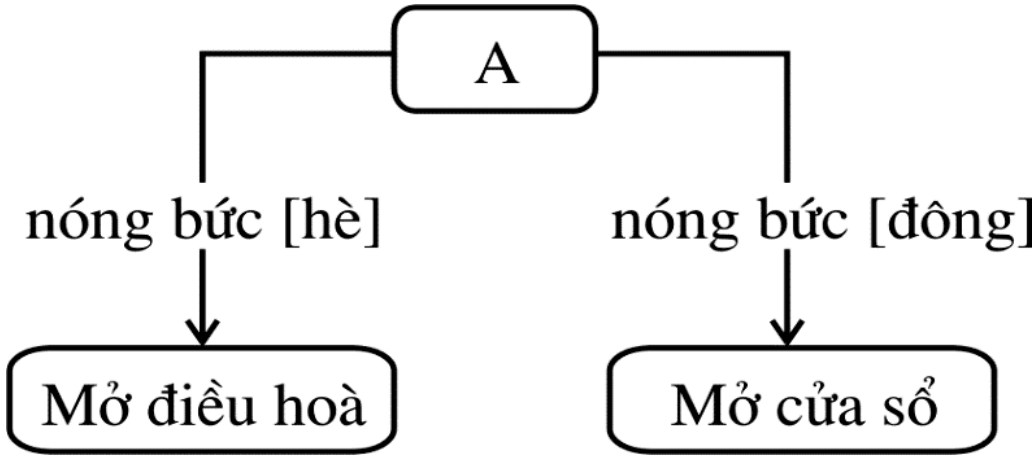


# Các dịch chuyển (2)

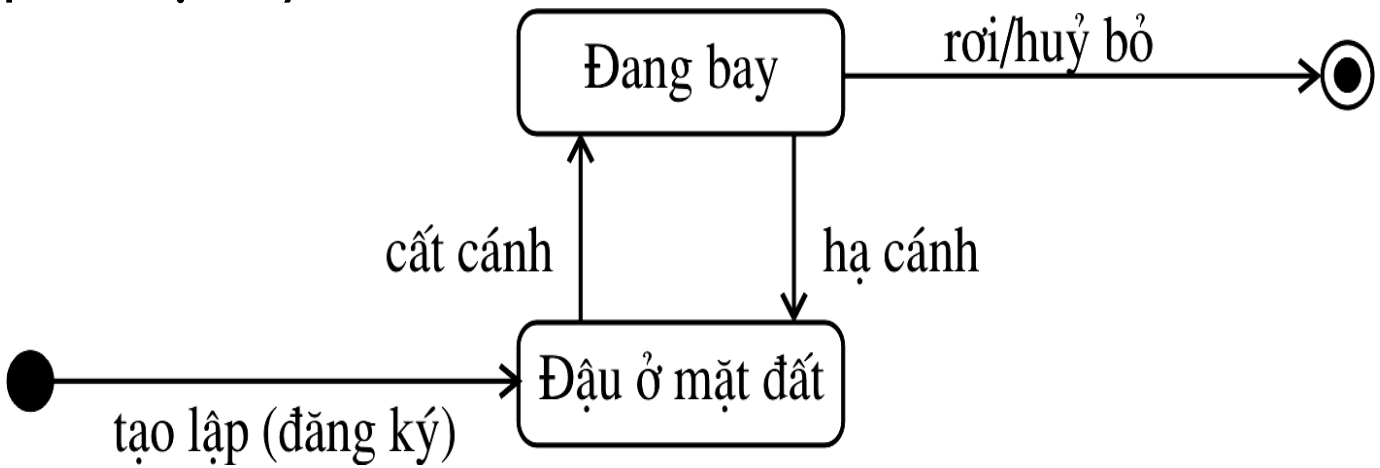
- Dịch chuyển trạng thái xảy ra
  - không chỉ vì sự xuất hiện của sự kiện kích thích
  - mà còn đòi hỏi thêm một điều kiện phải được thoả mãn
- Điều kiện kèm theo đó được gọi là một cảnh giới (guard),
  - Người ta thường dùng cảnh giới để làm cho ô-tô-mát (dịch chuyển trạng thái) trở nên xác định
- Biểu diễn được viết tiếp theo với sự kiện, trong một cặp ngoặc vuông



- ◦ Điều kiện bảo vệ (Guard Condition):
  - Là một biểu thức logic (đúng/sai) phải được thỏa mãn thì sự chuyển đổi mới được phép xảy ra, ngay cả khi sự kiện đã diễn ra.
  - Ví dụ,
    - sự kiện Khách hàng thanh toán xảy ra,
    - nhưng chỉ khi [Thanh toán hợp lệ] thì đơn hàng mới chuyển sang trạng thái Đã thanh toán.
- Biểu thức hành động (Action-Expression):
  - Là một tiến trình không thể bị ngắt quãng, được thực thi khi quá trình chuyển đổi diễn ra.
  - Ví dụ,
    - khi đơn hàng chuyển sang trạng thái Đã vận chuyển,
    - một hành động / Gửi email thông báo cho khách hàng sẽ được thực hiện.

# dịch chuyển đặc biệt

- là các dịch chuyển liên quan tới trạng thái vào hay trạng thái kết thúc:
  - Dịch chuyển từ trạng thái vào phải được kích thích bởi sự kiện tạo lập đối tượng
  - Dịch chuyển đến một trạng thái kết thúc phải đồng thời huỷ bỏ đối tượng
- Ví dụ:
  - Biểu diễn vòng đời của một đối tượng máy bay.
  - Sự kiện tạo lập cho phép đăng ký một máy bay.
  - Khi bị rơi, đối tượng máy bay đó phải bị huỷ bỏ.



# Sơ đồ Máy trạng thái Hành vi (Behavioral State Machine)

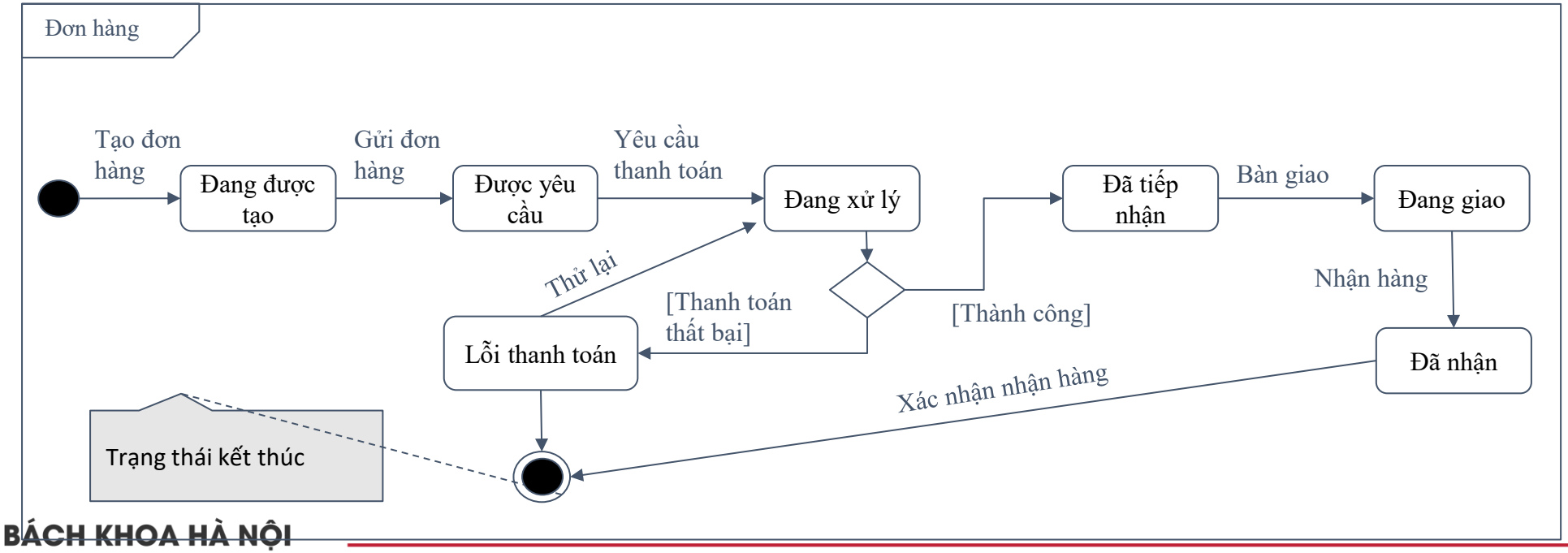
- Một mô hình động cho thấy các trạng thái (states) khác nhau mà một đối tượng duy nhất trải qua trong suốt vòng đời của nó để phản ứng với các sự kiện (events).
- Mục đích của việc MHH sự ứng xử là
  - **mô tả cách phản ứng của các đối tượng chủ động trước các sự kiện (thông điệp) được gửi đến**
- Để mô hình hóa vòng đời của một đối tượng phức tạp, chúng ta cần một công cụ đặc biệt.
  - Đó chính là Mô hình Trạng thái (State Models) với Máy trạng thái
  - và công cụ chính để thể hiện nó là Sơ đồ Máy trạng thái Hành vi (Behavioral State Machine)

# Behavioral State Machine

- Behavioral State Machine
  - Thường được sử dụng biểu diễn các đối tượng phức tạp có các trạng thái quan trọng cần theo dõi.
  - Nó  kể câu chuyện về một đối tượng phức tạp thay đổi qua các trạng thái khác nhau như thế nào khi có sự kiện xảy ra."
- Ví dụ:
  - một đối tượng Order (Đơn hàng) có thể có các trạng thái
    - Đang chờ xử lý,
    - Đã xác nhận,
    - Đang giao,
    - Đã hoàn thành.

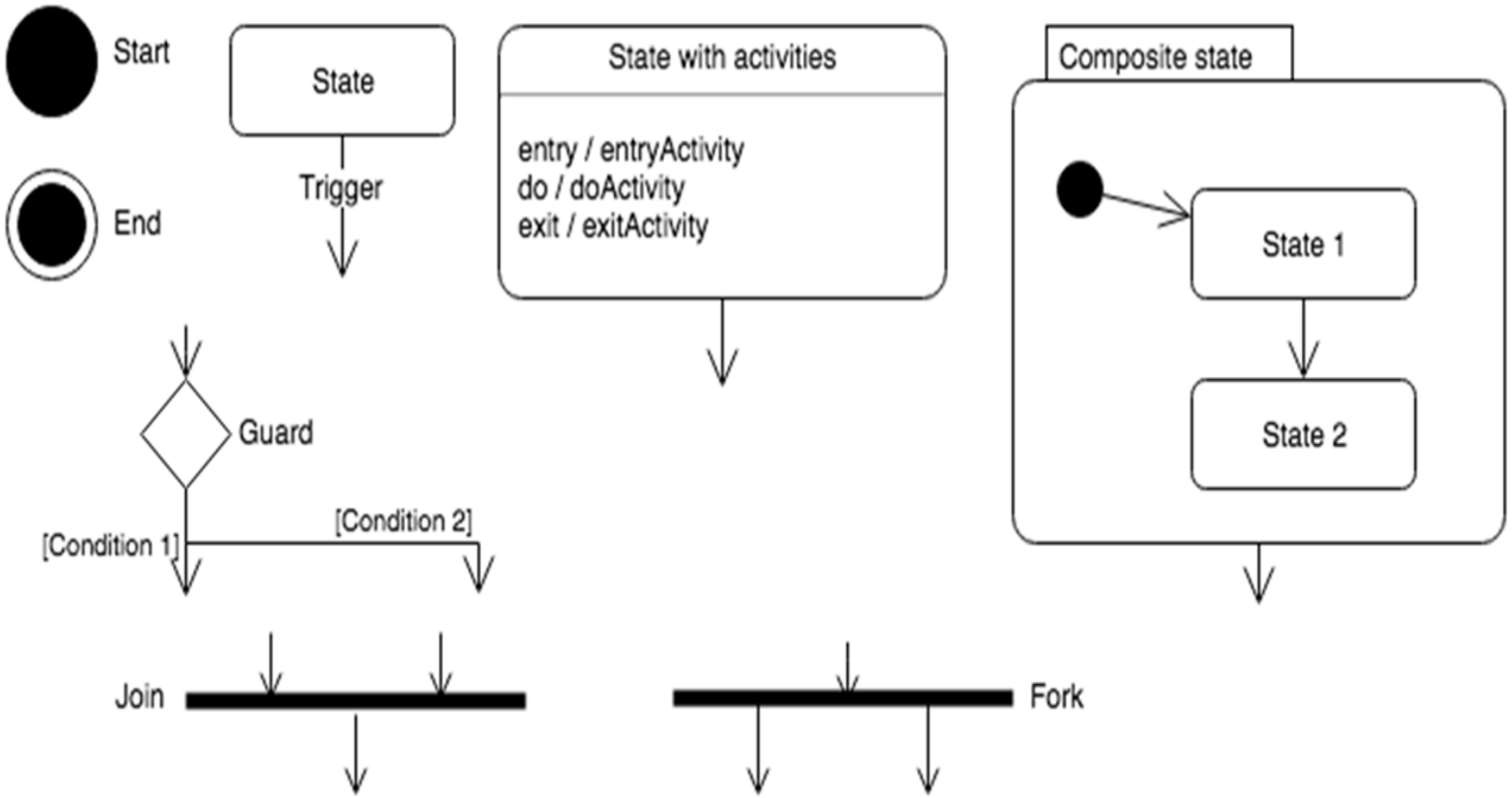
# State Chart Diagram - UML

- Biểu đồ máy trạng thái (UML là State Chart Diagram)
  - là một đồ thị có hướng, trong đó mỗi nút là một trạng thái,
  - mỗi cung là một dịch chuyển trạng thái
  - diễn tả quy luật thay đổi trạng thái và hành vi (ứng xử) của một đối tượng tuỳ thuộc vào các sự kiện được gửi đến



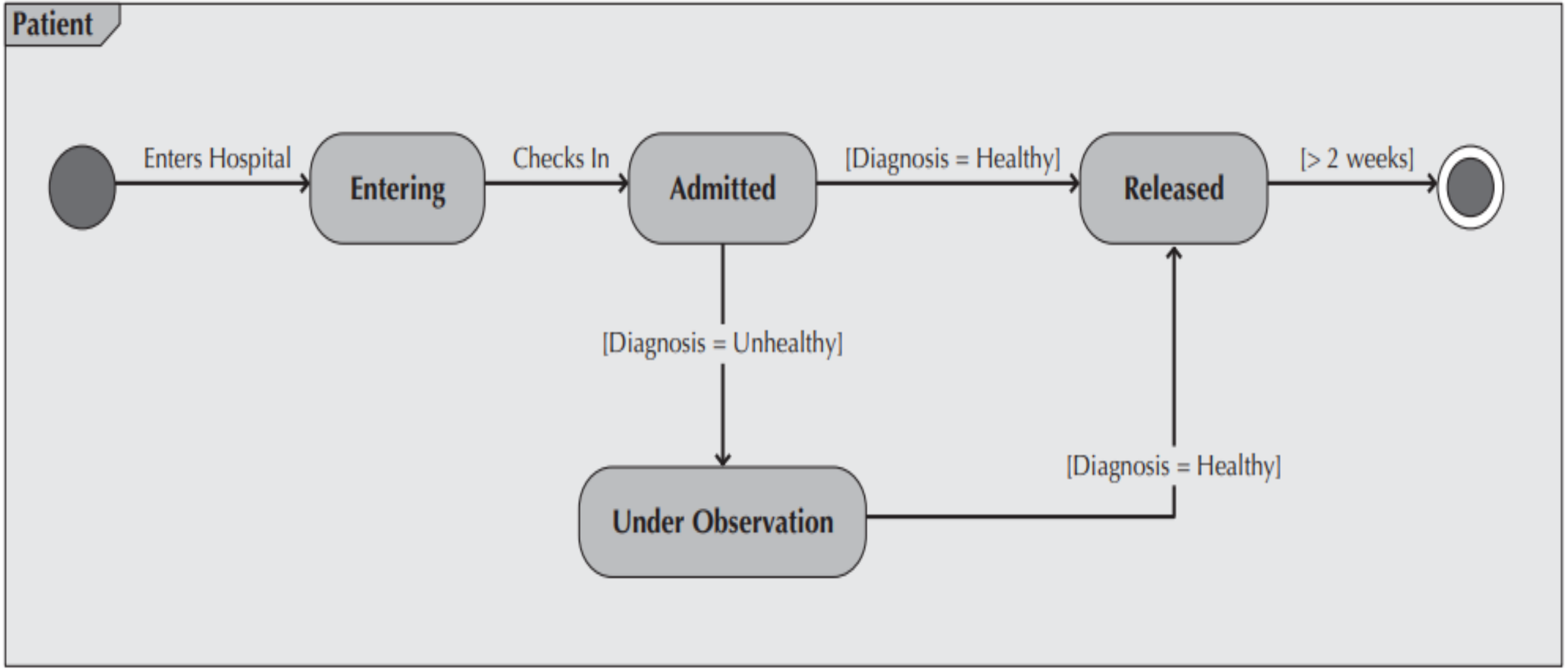
# Thành phần của Sơ đồ Máy trạng thái Hành vi

- State (Trạng thái):
  - Một điều kiện hoặc tình huống trong vòng đời của một đối tượng.
- Initial State
  - (Trạng thái ban đầu)
- Final State
  - (Trạng thái kết thúc)
- Event
  - (Sự kiện)
- Transition
  - Chuyển đổi
  - Cú pháp chuyển đổi:
  - transition-name (parameters, ...) [guard-condition] / action-expression.



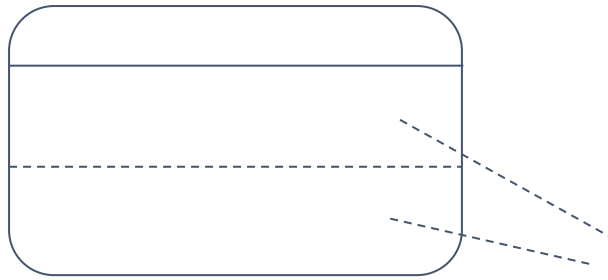
# Ví dụ Sơ đồ Máy trạng thái cho đối tượng "Patient"

- "Hãy xem xét một ví dụ thực tế.
  - Một bệnh nhân trong bệnh viện có thể trải qua nhiều trạng thái khác nhau, từ lúc nhập viện cho đến lúc xuất viện.
- Giải thích:
  - Bệnh nhân bắt đầu ở trạng thái Entering (Đang vào).
  - Khi sự kiện Checks In xảy ra, bệnh nhân chuyển sang trạng thái Admitted (Đã nhập viện).
  - Từ Admitted, nếu [Diagnosis = Healthy], bệnh nhân chuyển sang Released (Đã xuất viện).
  - Nếu [Diagnosis = Unhealthy], bệnh nhân chuyển sang Under Observation (Đang theo dõi).


Patient
Enters Hospital
Entering
Checks In
Admitted
[Diagnosis = Healthy]
Released
[> 2 weeks]
[Diagnosis = Unhealthy]
Under Observation
[Diagnosis = Healthy]

# Phân loại trạng thái

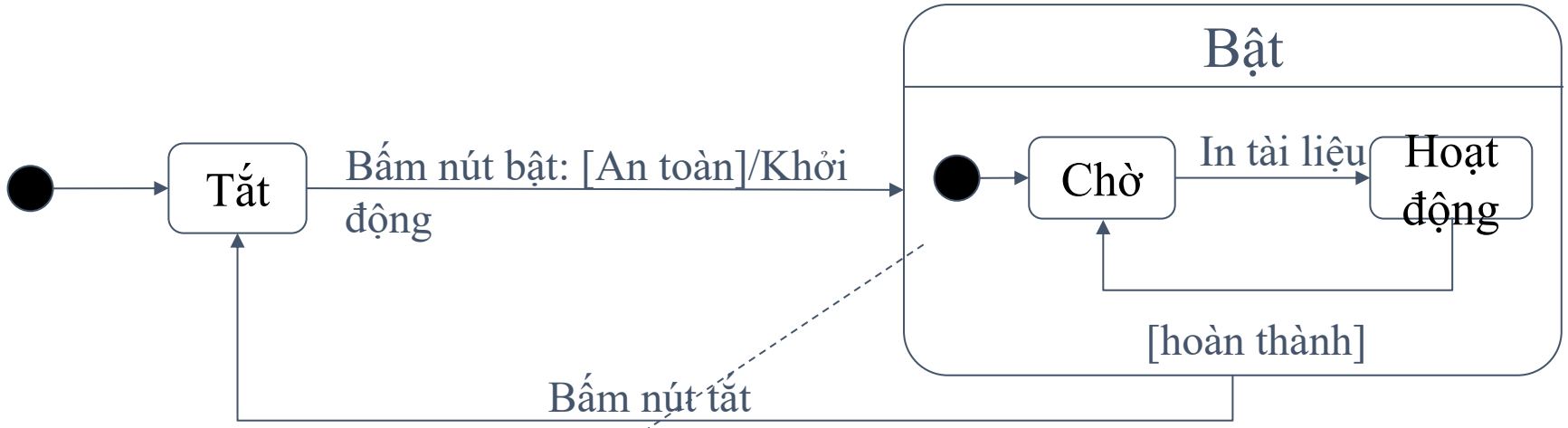
1. Trạng thái đơn:
   - Không chứa các thành phần bên trong
2. Trạng thái tổng hợp:
   - Bao gồm nhiều trạng thái con
   - Có thể bao gồm máy trạng thái hoàn chỉnh
     - Có bắt đầu và kết thúc riêng.
     - Tạo thành các máy trạng thái lồng nhau
   - Có thể bao gồm 1 vùng hoặc nhiều vùng song song
   - Các luồng trạng thái song song
     - được gọi là các trạng thái tổng hợp trực giao (orthogonal)
     - Được sở hữu bởi cùng 1 trạng thái hoặc cùng ở mức đỉnh của 1 máy trạng thái



…

*2 vùng trực giao tồn tại song song*

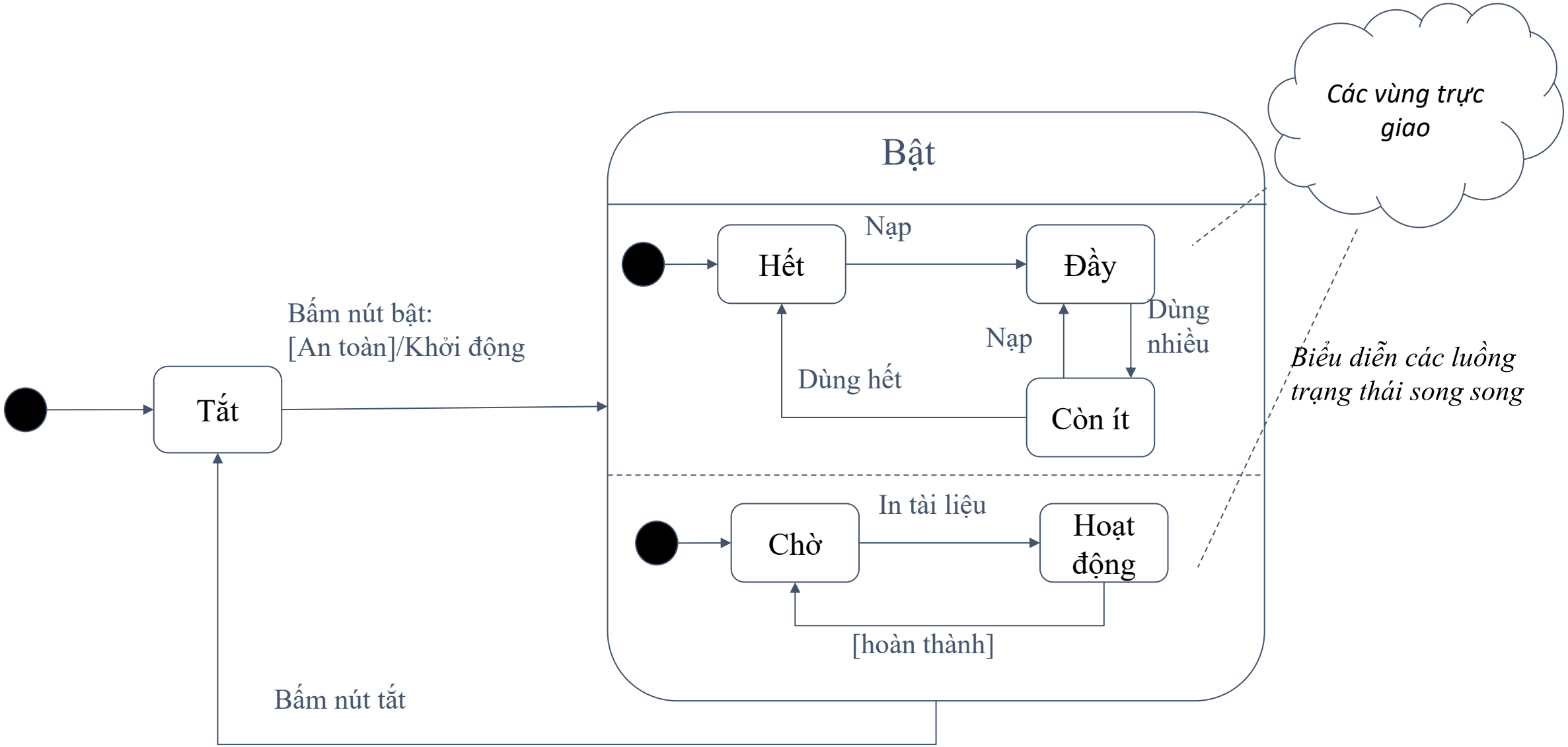
# Trạng thái tổng hợp với 1 vùng



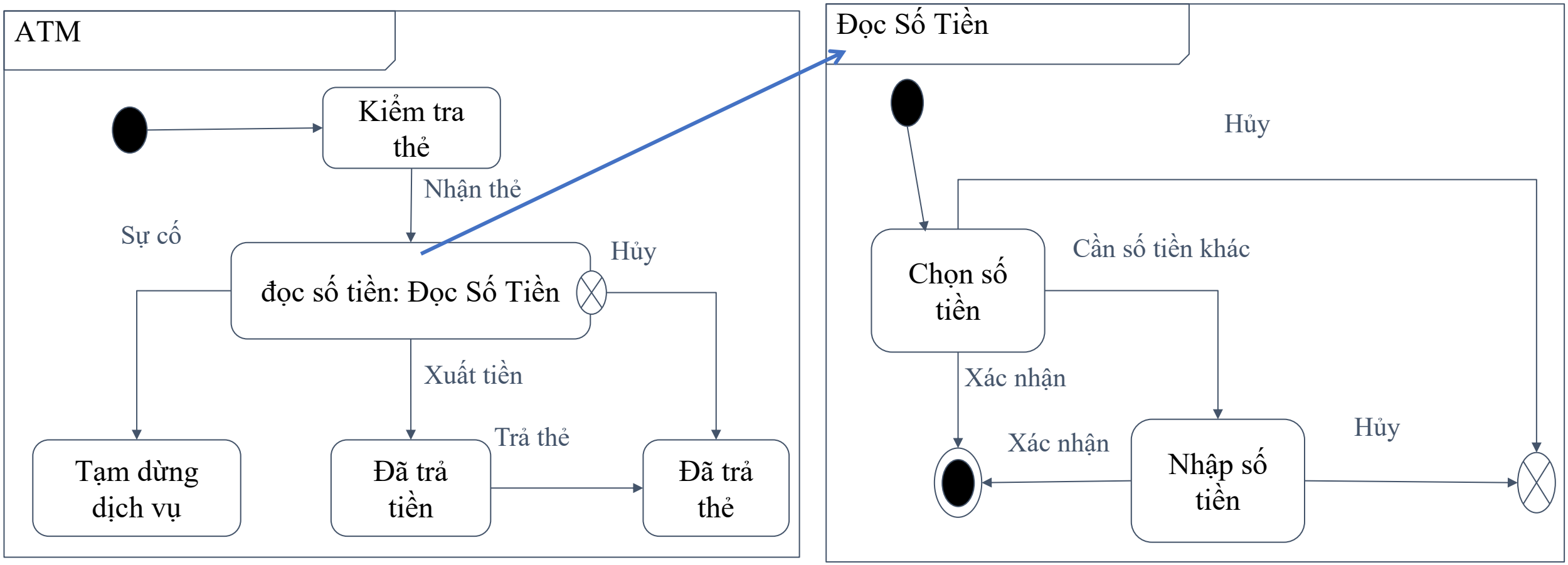

Trạng thái tổng hợp với 1 vùng

*Trong trạng thái bật, máy in có thể ở trạng thái Chờ hoặc Hoạt động.*

# Biểu diễn các luồng trạng thái song song



# Máy trạng thái lồng nhau



# Quy trình 5 bước Tạo Sơ đồ Máy trạng thái

- 1. Thiết lập Ngữ cảnh (Set Context):
  - Chọn một lớp (class) phức tạp để mô hình hóa.
- 2. Xác định các Trạng thái của Đối tượng (Identify Object States):
  - Liệt kê tất cả các trạng thái có thể có của đối tượng, bao gồm trạng thái bắt đầu và kết thúc.
- 3. Bố trí Sơ đồ (Lay Out Diagram):
  - Sắp xếp các trạng thái theo một trình tự logic, thường là từ trái sang phải hoặc từ trên xuống dưới.
- 4. Thêm các Chuyển đổi (Add Transitions):
  - Xác định các sự kiện, hành động và điều kiện bảo vệ gây ra sự chuyển đổi giữa các trạng thái và vẽ các mũi tên tương ứng.
- 5. Xác thực Sơ đồ (Validate):
  - Đảm bảo mỗi trạng thái đều có thể đạt được và có thể thoát ra (trừ trạng thái kết thúc).

# Tóm tắt State model

- Mục đích:
  - Để biểu diễn các trạng thái một vật thể và các sự kiện khiến vật thể đó chuyển từ trạng thái này sang trạng thái khác.
  - Tập trung: Hành vi bên trong và vòng đời của một đối tượng duy nhất.
- Ưu điểm:
  - Hành vi chi tiết: Cung cấp cái nhìn toàn diện về hành vi động của đối tượng theo thời gian.
  - Logic phức tạp: Lý tưởng để mô hình hóa các đối tượng có logic nội bộ hoặc vòng đời phức tạp.
- Nhược điểm:
  - Ít tập trung vào sự hợp tác: Không hiển thị rõ ràng cách đối tượng này tương tác với các thành phần khác.
  - Trừu tượng hóa: Có thể trừu tượng hóa một số tương tác cấp cao có vai trò quan trọng đối với thiết kế trải nghiệm người dùng.

# Interaction Models

# Giới thiệu

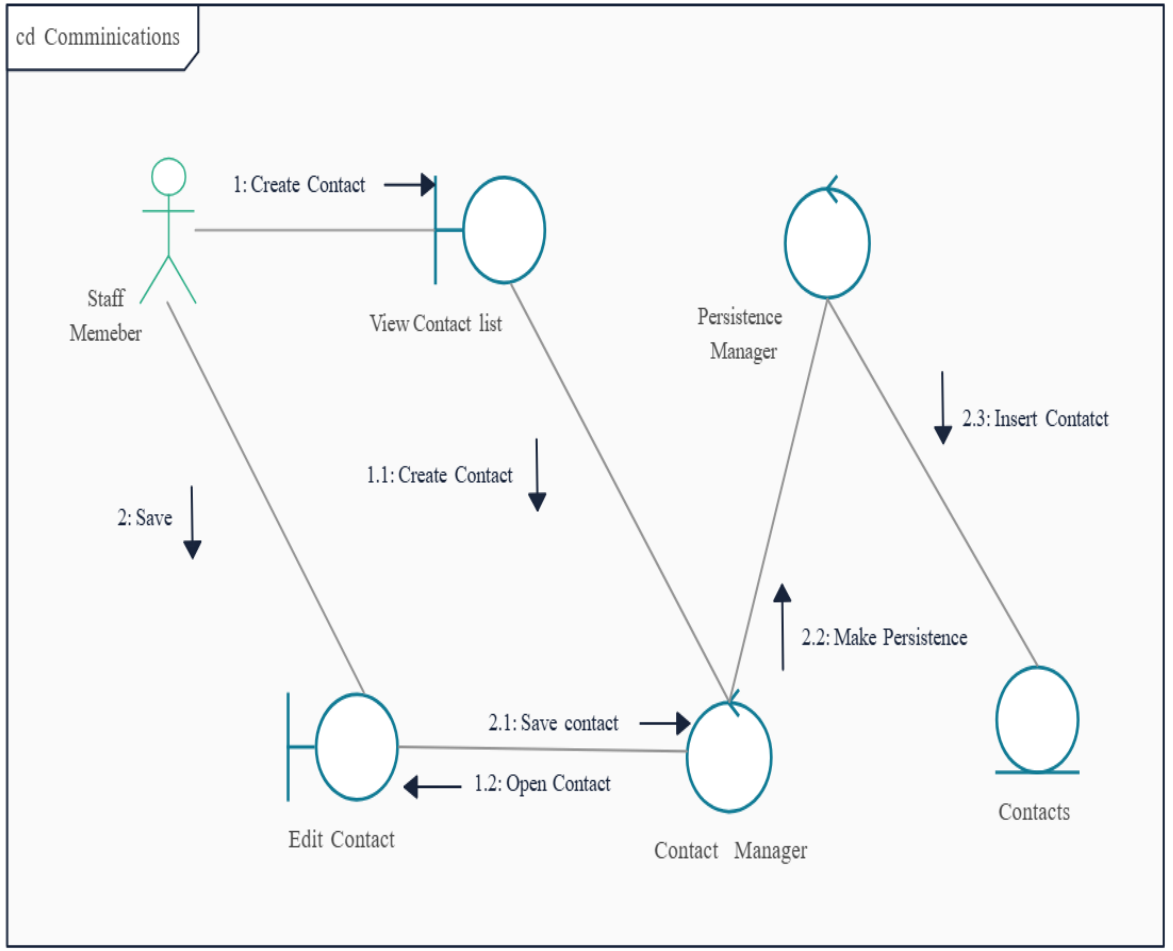
- Mô hình Trạng thái (State Models),
  - một công cụ mạnh giúp kể câu chuyện về "vòng đời" của một đối tượng duy nhất.
  - Nó cho thấy một đối tượng thay đổi qua các trạng thái khác nhau như thế nào để phản ứng với các sự kiện.
- Tuy nhiên, trong một hệ thống thực tế,
  - các đối tượng không tồn tại một cách biệt lập.
  - chúng là một xã hội thu nhỏ, liên tục hợp tác, trao đổi thông tin để cùng nhau hoàn thành một công việc lớn hơn.
- Nếu Mô hình Trạng thái là "thế giới nội tâm" của một đối tượng,
  - Cái gì dùng để mô tả về Giao tiếp hay "cuộc đối thoại" của các đối tượng.

# (Interaction Models)

- Mô hình Tương tác
  - là công cụ thiết yếu để mô hình hóa khía cạnh động (dynamic aspect) của hệ thống.
  - thể hiện cách các đối tượng giao tiếp thông qua thông điệp để thực hiện các chức năng
- Nó giúp chuyển từ việc hiểu yêu cầu sang
  - thiết kế cách các đối tượng phần mềm hợp tác với nhau để hiện thực hóa các yêu cầu đó.

# Interaction Models trả lời các câu hỏi quan trọng

- những đối tượng nào cần tham gia
  - Để hoàn thành một use case,?
- Chúng gửi những thông điệp (messages) gì cho nhau?
- Các thông điệp đó được gửi theo trình tự nào?



# Các thành phần chính

- Để xây dựng một mô hình tương tác hiệu quả, cần phải xác định một số yếu tố cốt lõi:
  - Actors/Agents Các tác nhân/đại lý:
    - Các thực thể bên ngoài (người dùng hoặc các hệ thống khác) khởi tạo hoặc phản hồi các tác nhân kích hoạt.
  - Objects/Roles  Đối tượng/Vai trò:
    - Các thành phần nội bộ trong hệ thống thực thi logic.
  - Messages Thông điệp:
    - Là những thông tin được truyền tải (yêu cầu, phản hồi hoặc tín hiệu) .
  - Flow of Control Luồng điều khiển:
    - Trình tự hoặc logic điều kiện quy định thông điệp nào được gửi đi và khi nào.
  - Links:
    - Các đường dẫn vật lý hoặc liên kết cho phép thông điệp di chuyển giữa các đối tượng.

# So sánh Đối tượng (Object) và Tác nhân (Actor)

- Tác nhân (Actor) là một người hoặc một hệ thống bên ngoài khởi xướng hoặc tham gia vào kịch bản.
- Đối tượng (Object) là các thực thể bên trong hệ thống hợp tác với nhau để thực hiện công việc.

- Hãy hình dung các tác nhân và đối tượng
  - như những "diễn viên" trong một vở kịch.
  - Có diễn viên chính (tác nhân khởi xướng) và các diễn viên phụ (các đối tượng bên trong).
- Ví dụ,
  - trong kịch bản "Đặt lịch hẹn",
    - BenhNhan (Patient) là tác nhân,
    - còn LichHen (Appointment) và LeTan (Receptionist) là các đối tượng.

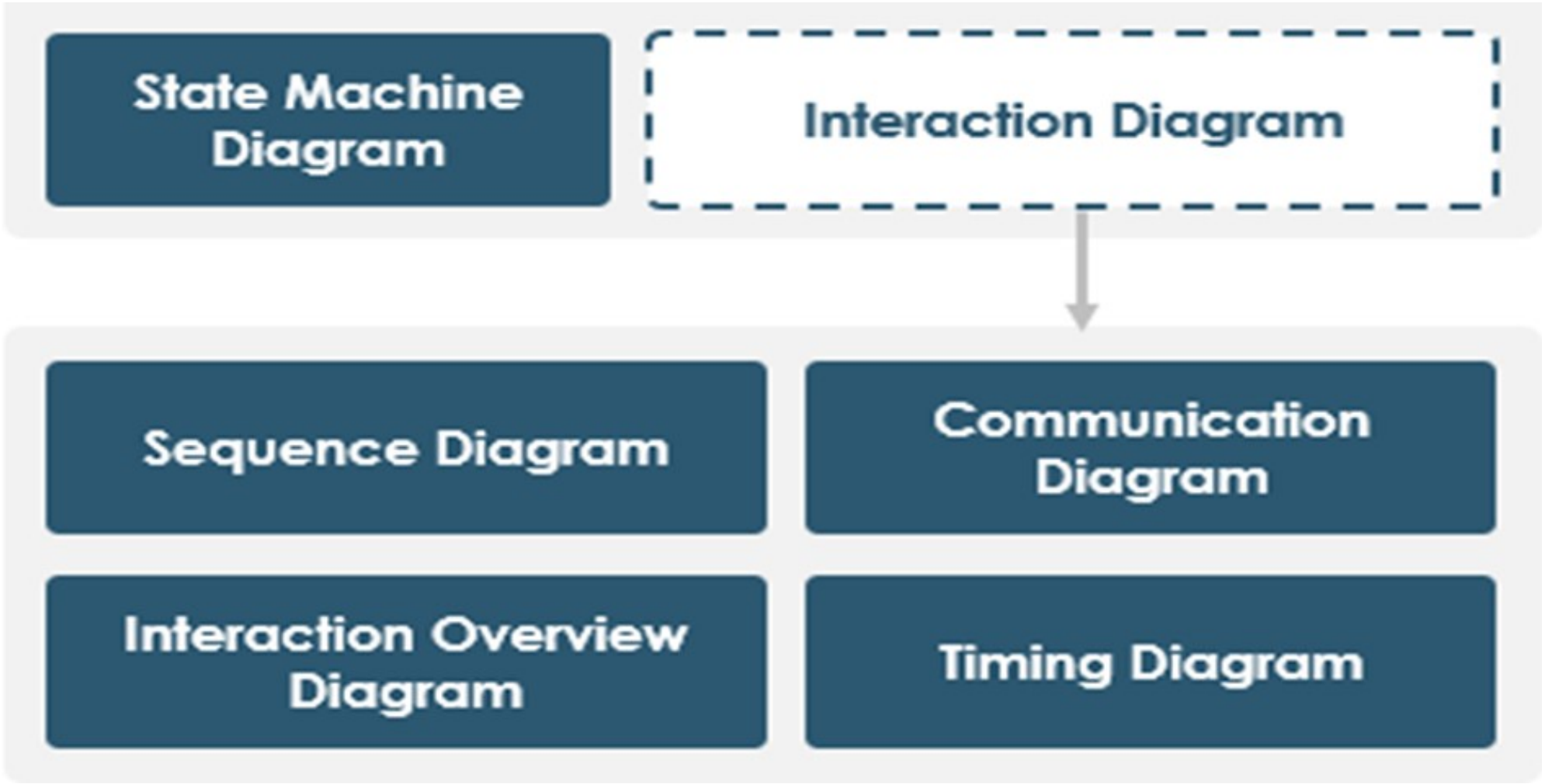
# Liên kết và thông điệp

- 1.Liên kết (Link)
  - là một kết nối giữa các đối tượng, thể hiện rằng chúng có khả năng giao tiếp với nhau.
  - Liên kết giống như một "kênh liên lạc" giữa các diễn viên.
    - Nếu hai đối tượng có một đường liên kết, chúng có thể "nói chuyện" được với nhau.
- 2. Thông điệp (Message)
  - là sự giao tiếp từ một đối tượng (source) đến (destination),
  - nó là một lời gọi hàm hoặc thủ tục từ đối tượng này đến đối tượng khác.
- Thông điệp chính là "lời thoại" trong vở kịch. Link là quan hệ để các diễn viên có thể truyền tải Message
  - Ví dụ, đối tượng LeTan gửi thông điệp kiemTraLichTrong() đến đối tượng LichHen.

# Core Interaction Modeling Diagrams

- Communication (Collaboration) Diagrams:
  - Nhấn mạnh tổ chức cấu trúc của các đối tượng và các liên kết tạo điều kiện giao tiếp của chúng.
- Sequence Diagrams:
  - Tập trung vào thứ tự thời gian của việc trao đổi tin nhắn giữa các đối tượng cho các trường hợp sử dụng cụ thể.
- Interaction Overview Diagrams:
  - chế độ xem cấp cao kết hợp sơ đồ hoạt động và sơ đồ trình tự để hiển thị cách các tương tác khác nhau kết hợp.
- Timing Diagrams:
  - Tập trung vào các ràng buộc thời gian cụ thể và thay đổi trạng thái của các đối tượng trong một dòng thời gian tuyến tính.

# Interaction Model



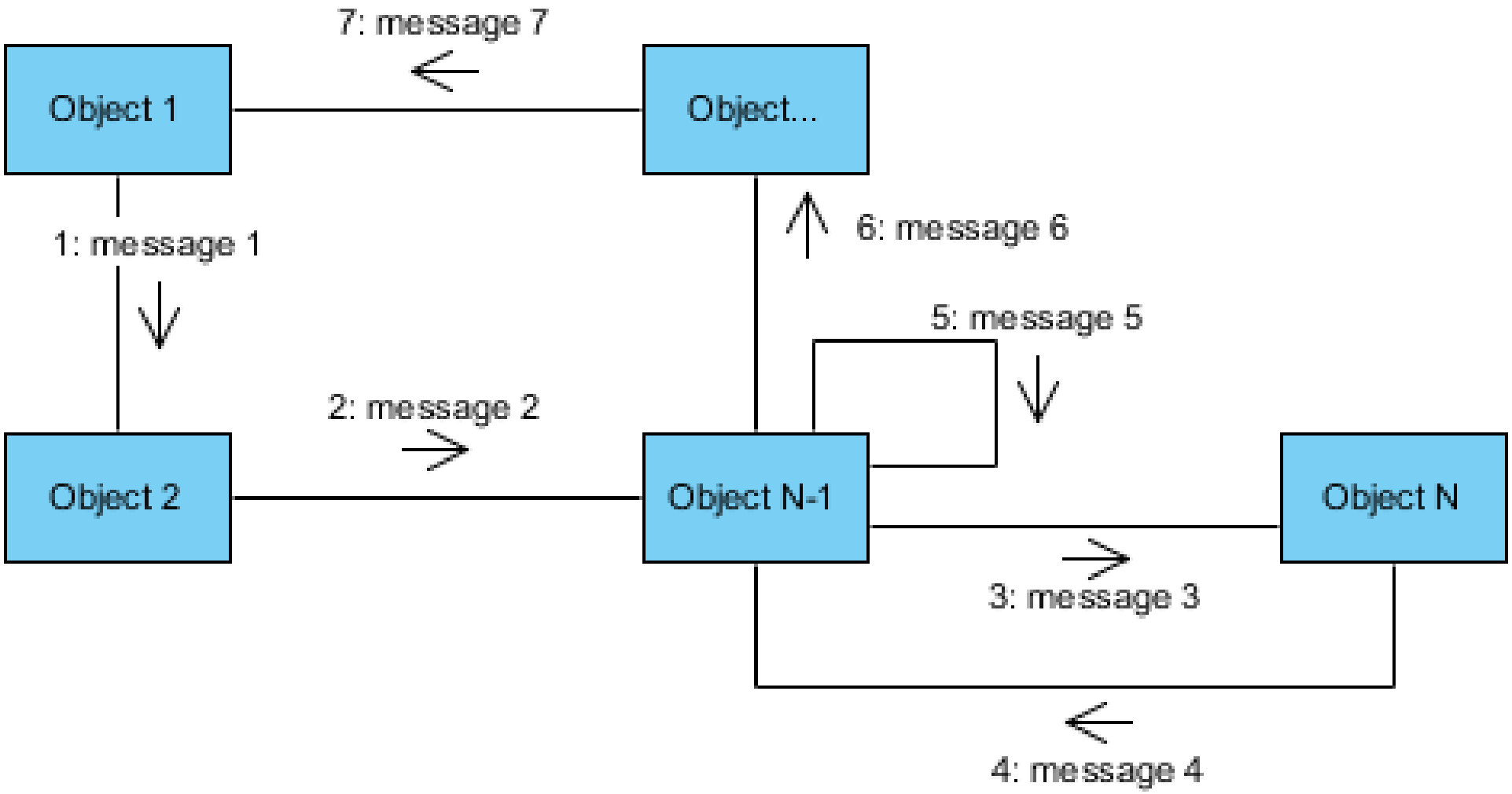
# 1.Comunication Diagram - Sơ đồ giao tiếp

- Comunication Diagram. (Collaboration Diagram Uml1.x)
  - Là sơ đồ minh họa cách các đối tượng phối hợp với nhau để thực hiện một nhiệm vụ cụ thể
  - nó tập trung vào cấu trúc tổ chức và các liên kết giữa các đối tượng hơn là dòng thời gian theo trình tự thời gian.
- Chúng cũng hữu ích trong việc kết hợp và phân tích hành vi của một hệ thống trong quá trình thiết kế và phân tích bằng cách
  - chỉ ra cách chúng hoạt động cùng nhau để đạt được mục tiêu cuối cùng của hệ thống

# Hoạt động

- Một mô hình động cho thấy cách các đối tượng hợp tác với nhau để thực hiện một use case hoặc một kịch bản.
- Về cơ bản là phần mở rộng của biểu đồ đối tượng, thêm thông tin tin nhắn vào cấu trúc đối tượng tĩnh.
- Nhấn mạnh vào luồng thông điệp qua các đối tượng (flow of messages) và sự phụ thuộc lẫn nhau của chúng.

- "Nếu Sơ đồ Chuỗi giống như một kịch bản phim,
  - thì Sơ đồ Giao tiếp giống như một sơ đồ mạng lưới,
- Cho thấy 'ai nói chuyện với ai'.
  - Nó không tập trung vào thời gian mà tập trung vào các mối liên kết cấu trúc giữa các đối tượng."

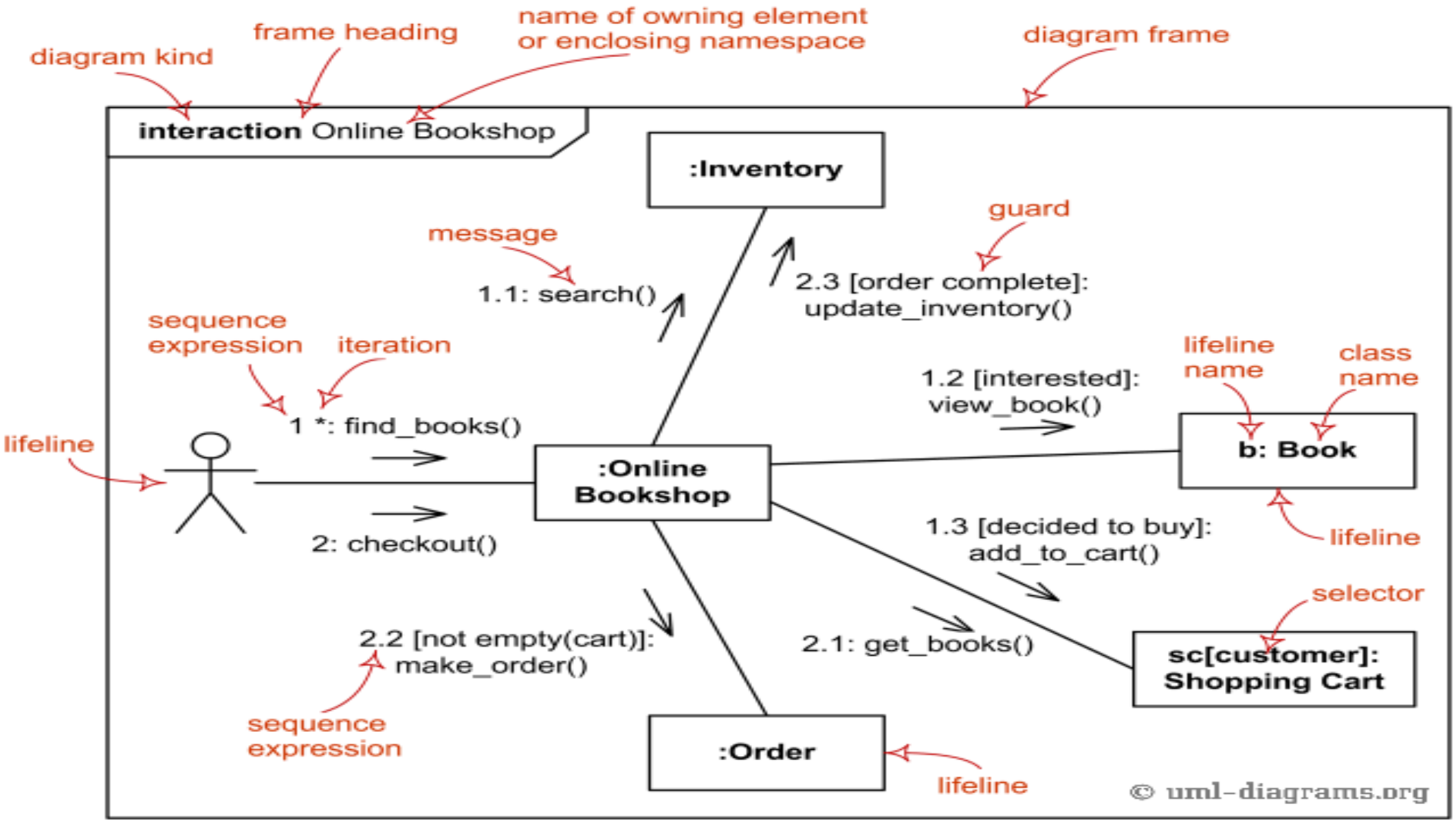

7: message 7
Object 1
Object...
1: message 1
6: message 6
5: message 5
2: message 2
Object 2
Object N-1
Object N
3: message 3
4: message 4

# Đặc điểm chính

- Mối quan hệ giữa các đối tượng:
  - minh họa cách các đối tượng tương tác với nhau thông qua các thông điệp gửi và nhận.
- Tin nhắn được đánh số:
  - gắn nhãn theo số thứ tự thời gian (ví dụ: 1.0, 2.1) để chỉ ra thứ tự của chúng trong quá trình tương tác.
- Biểu diễn đối tượng/tác nhân:
  - Họ sử dụng các đường liên kết (tương tự như sơ đồ đối tượng) để biểu diễn các đối tượng hoặc tác nhân trong hệ thống.
- Tập trung vào hành vi và cấu trúc:
  - Các sơ đồ kết hợp các khía cạnh của cấu trúc tĩnh (như sơ đồ đối tượng) và hành vi động (như sơ đồ trình tự), hiển thị cả những yếu tố nào tương tác với nhau và cách chúng tương tác.

# Các Thành phần của Sơ đồ Giao tiếp

- Objects (Lifelines): Được thể hiện dưới dạng hình chữ nhật hiển thị các bên tham gia.
- Links: mối quan hệ hoặc đường dẫn giao tiếp.
- Messages: nội dung giao tiếp.
- Sequence Numbers: trật tự trong bố cục
  - Được ghi nhãn trên các đường liên kết.
    - Đánh số tuần tự (Sequence Number): Rất quan trọng để thể hiện thứ tự. Sử dụng hệ thống phân cấp dạng Dewey (ví dụ: 1, 1.1, 1.2, 2).
    - Có điều kiện (Conditional): Một điều kiện (guard) được đặt trong ngoặc vuông [] trước số thứ tự.
    - Lặp lại (Iteration): Một dấu hoa thị * được đặt sau số thứ tự.


diagram kind
frame heading
name of owning element or enclosing namespace
diagram frame
interaction Online Bookshop
:Inventory
guard
message
1.1: search()
2.3 [order complete]: update_inventory()
sequence expression
iteration
1 *: find_books()
1.2 [interested]: view_book()
lifeline name
class name
b: Book
lifeline
:Online Bookshop
2: checkout()
1.3 [decided to buy]: add_to_cart()
lifeline
selector
2.2 [not empty(cart)]: make_order()
2.1: get_books()
sc[customer]: Shopping Cart
sequence expression
:Order
lifeline
© uml-diagrams.org

# So sánh Sơ đồ Chuỗi và Sơ đồ Giao tiếp

- Giống nhau:
  - Cả hai đều là sơ đồ tương tác (Interaction Diagrams) và về mặt ngữ nghĩa là tương đương (có thể chuyển đổi qua lại).
- Khác nhau:
  - Sơ đồ Chuỗi:
    - Nhấn mạnh thời gian (time ordering).
    - Dễ thấy thứ tự thông điệp.
    - Tốn nhiều không gian chiều ngang.
  - Sơ đồ Giao tiếp:
    - Nhấn mạnh không gian/liên kết (space/links).
    - Khó thấy thứ tự thông điệp hơn nhưng tiết kiệm không gian hơn.
- Kết luận
  - "Cả hai sơ đồ đều mô tả sự tương tác, nhưng chúng làm nổi bật những khía cạnh khác nhau.
  - Việc hiểu rõ sự khác biệt này sẽ giúp chúng ta chọn đúng công cụ cho đúng mục đích."

# Khi nào nên sử dụng sơ đồ giao tiếp

- Tập trung vào cấu trúc:
  - Sử dụng phương pháp này khi bạn cần xem "ai nói chuyện với ai" và
  - sự liên kết tổng thể của một hệ thống.
- Chế độ xem thu gọn:
  - Chế độ này thường chiếm ít không gian hơn so với sơ đồ trình tự,
  - hữu ích cho việc tổng quan cấp cao hoặc các nhóm đối tượng phức tạp.
- Refining Architecture:
  - Công cụ lý tưởng để xác định các giao diện cần thiết và
  - những thay đổi cấu trúc trong giai đoạn thiết kế ban đầu.

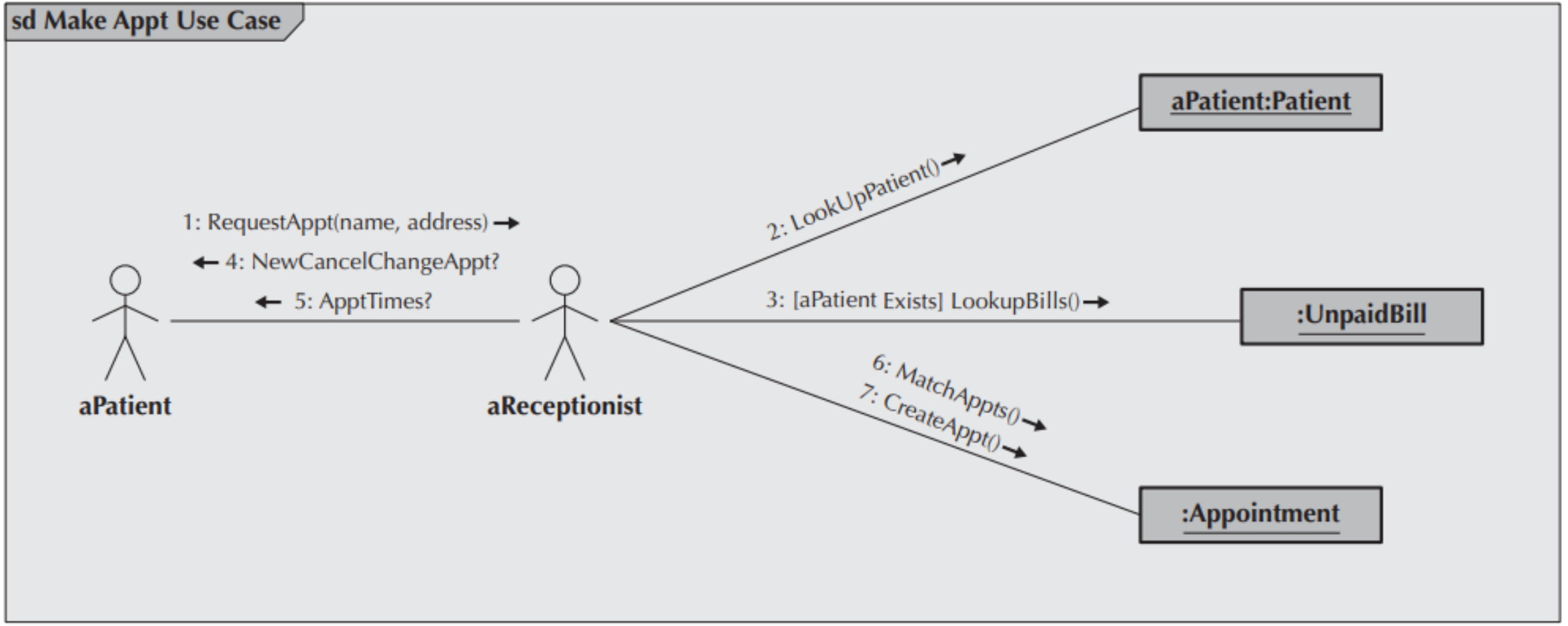
# Quy trình Tạo Sơ đồ Giao tiếp

- 1. Thiết lập Ngữ cảnh (Set Context):
  - Xác định sơ đồ này dành cho use case hay kịch bản nào.
- 2. Xác định Đối tượng, Tác nhân và Liên kết (Identify Objects, Actors, & Associations):
  - Xác định các đối tượng/tác nhân tham gia và các liên kết giữa chúng.
- 3. Bố trí Sơ đồ (Lay Out Diagram):
  - Sắp xếp các đối tượng/tác nhân để giảm thiểu các đường liên kết cắt chéo nhau.
- 4. Thêm các Thông điệp (Add Messages):
  - Ghi nhãn các liên kết với tên thông điệp và số thứ tự phù hợp.
- 5. Xác thực Sơ đồ (Validate):
  - Đảm bảo sơ đồ mô tả đầy đủ các bước trong kịch bản.

# Ví dụ Sơ đồ Giao tiếp cho Use Case "Make Old Patient Appt"

- "Hãy cùng xem lại kịch bản đặt lịch hẹn của bệnh nhân cũ, nhưng lần này chúng ta sẽ biểu diễn nó bằng Sơ đồ Giao tiếp để thấy rõ các liên kết."
- Giải thích các thông điệp theo số thứ tự:
  - 1: RequestAppt(name, address)
  - 2: LookUpPatient()
  - 3: [aPatient Exists] LookupBills()
  - 4: NewCancelChangeAppt?()
  - ◦ ... (các thông điệp còn lại theo thứ tự).

# Make Old Patient Appt



# Sequence Diagram

# 1. Sequence Diagram:

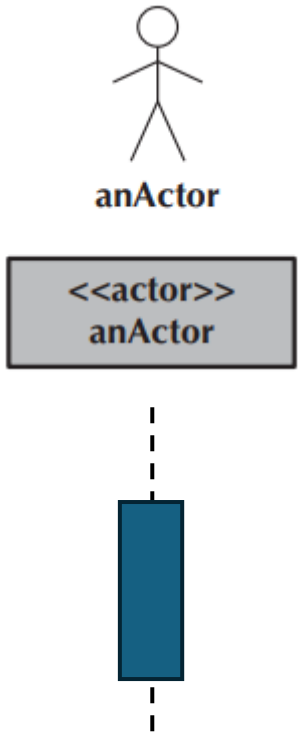
- Trình tự (Sequence)
  - Đây là thứ tự mà các thông điệp được gửi đi theo thời gian.
  - Trình tự là yếu tố cực kỳ quan trọng vì nó quyết định logic của toàn bộ quá trình tương tác.
- Ví dụ: Trình tự chính là thứ tự các lời thoại trong kịch bản.
  - Ai nói trước, ai nói sau sẽ quyết định toàn bộ diễn biến của câu chuyện.
  - Việc gửi thông điệp sai thứ tự có thể dẫn đến kết quả sai hoặc hệ thống không hoạt động đúng.

# Sequence Diagram - Biểu đồ trình tự

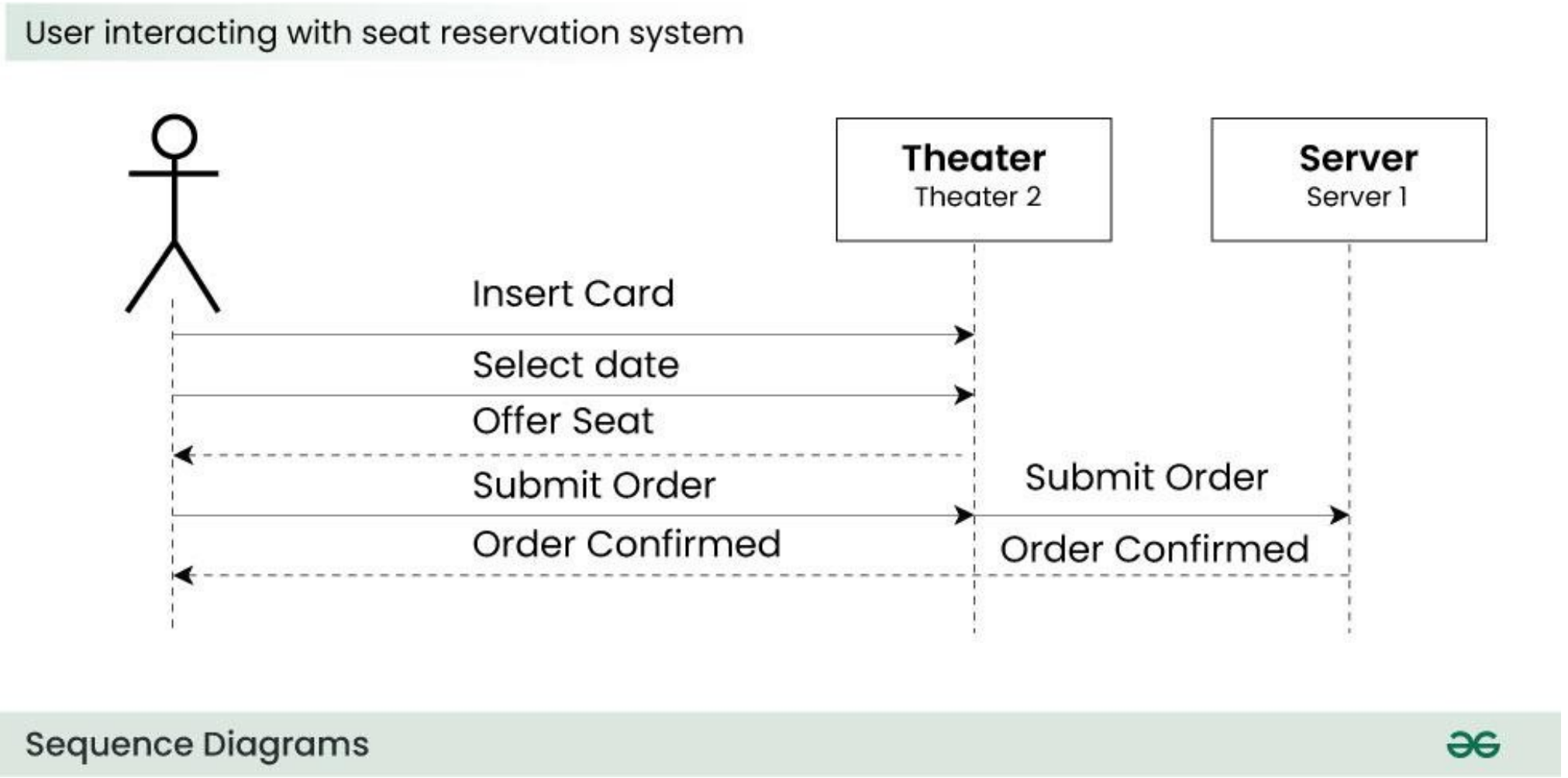
- Sequence Diagram:
  - là một loại biểu đồ UML thể hiện trực quan sự tương tác giữa các đối tượng hoặc thành phần theo thời gian.
- Chúng tập trung vào
  - thứ tự và thời gian của các thông điệp hoặc sự kiện được trao đổi giữa các thành phần của hệ thống.
- Biểu đồ này
  - nắm bắt cách các đối tượng giao tiếp với nhau thông qua một chuỗi thông điệp,
  - cung cấp cái nhìn rõ ràng về trình tự các thao tác hoặc quy trình.

# Các Thành phần của Sơ đồ Chuỗi  trong UML

- Actor (Tác nhân):
  - Được đặt ở phía trên cùng bên trái sơ đồ.
- Object (Đối tượng):
  - Được đặt trên cùng, bên phải Actor.
  - Ký hiệu: tên_đối_tượng:TênLớp.
- Lifeline (Đường sống):
  - Một đường nét đứt thẳng đứng bên dưới mỗi actor và object,
  - biểu thị sự tồn tại của chúng theo thời gian.
- Execution Occurrence (Sự kiện thực thi):
  - Một hình chữ nhật hẹp trên đường sống, cho biết khoảng thời gian đối tượng đang gửi hoặc nhận thông điệp (đang hoạt động).



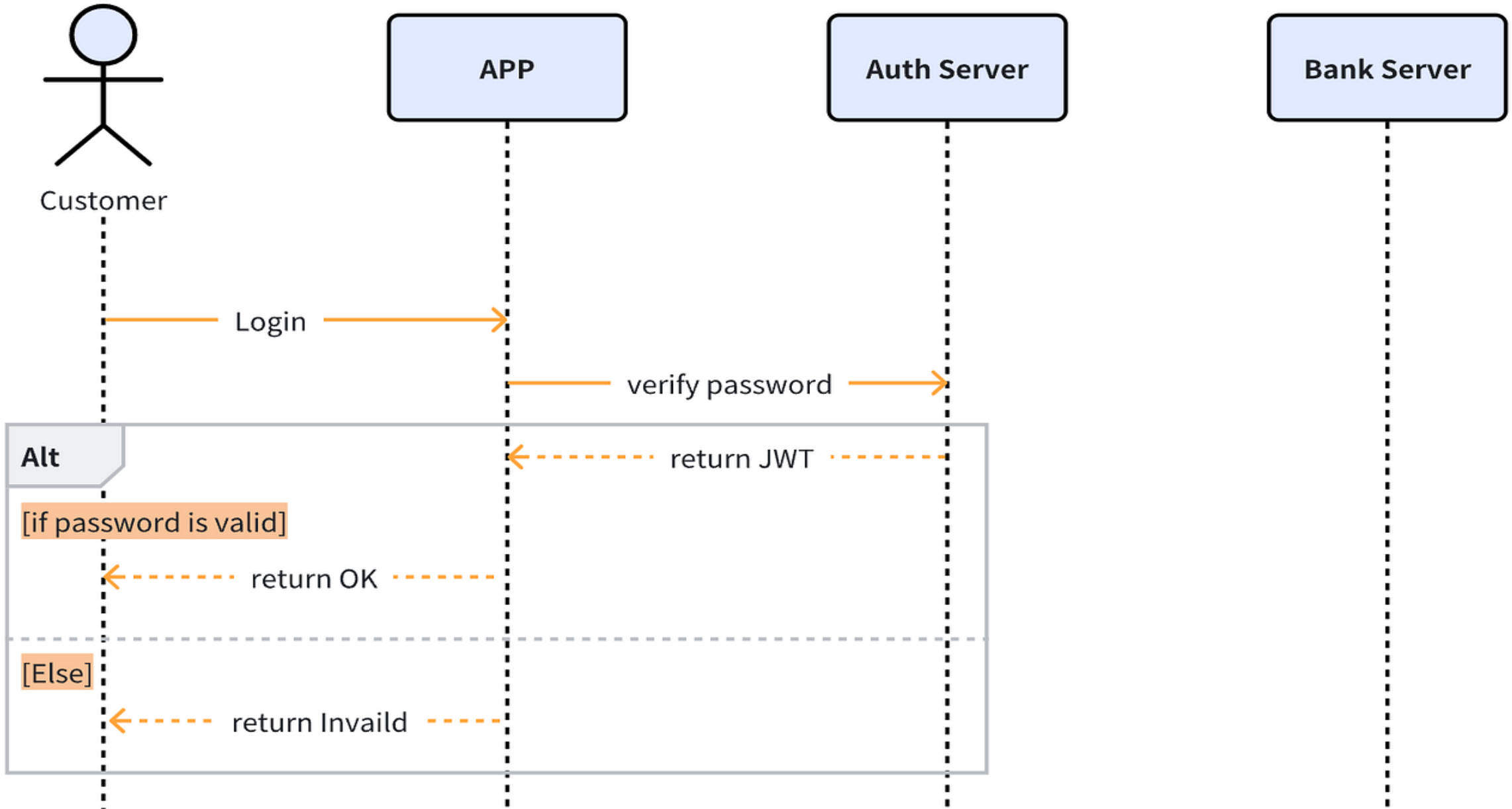
# Ví dụ Sơ đồ tuần tự



# Các Cấu trúc Điều khiển trong Sơ đồ Chuỗi: Frame

- "Một kịch bản không phải lúc nào cũng diễn ra tuần tự.
  - Sẽ có những lựa chọn, những hành động tùy chọn.
  - 'khung' (Frames) để mô hình hóa các logic phức tạp này."
- Frame (Khung):
  - Biểu thị các cấu trúc điều khiển phức tạp.
  - opt (Optional Frame):
    - Cho biết một thông điệp là tùy chọn, chỉ xảy ra nếu một điều kiện (guard) là đúng.
  - alt (Alternative Frame):
    - Dùng cho logic if-then-else. Khung được chia vùng, mỗi vùng có một điều kiện (guard) riêng.
  - loop (Loop Frame):
    - Cho biết một chuỗi thông điệp được lặp lại.

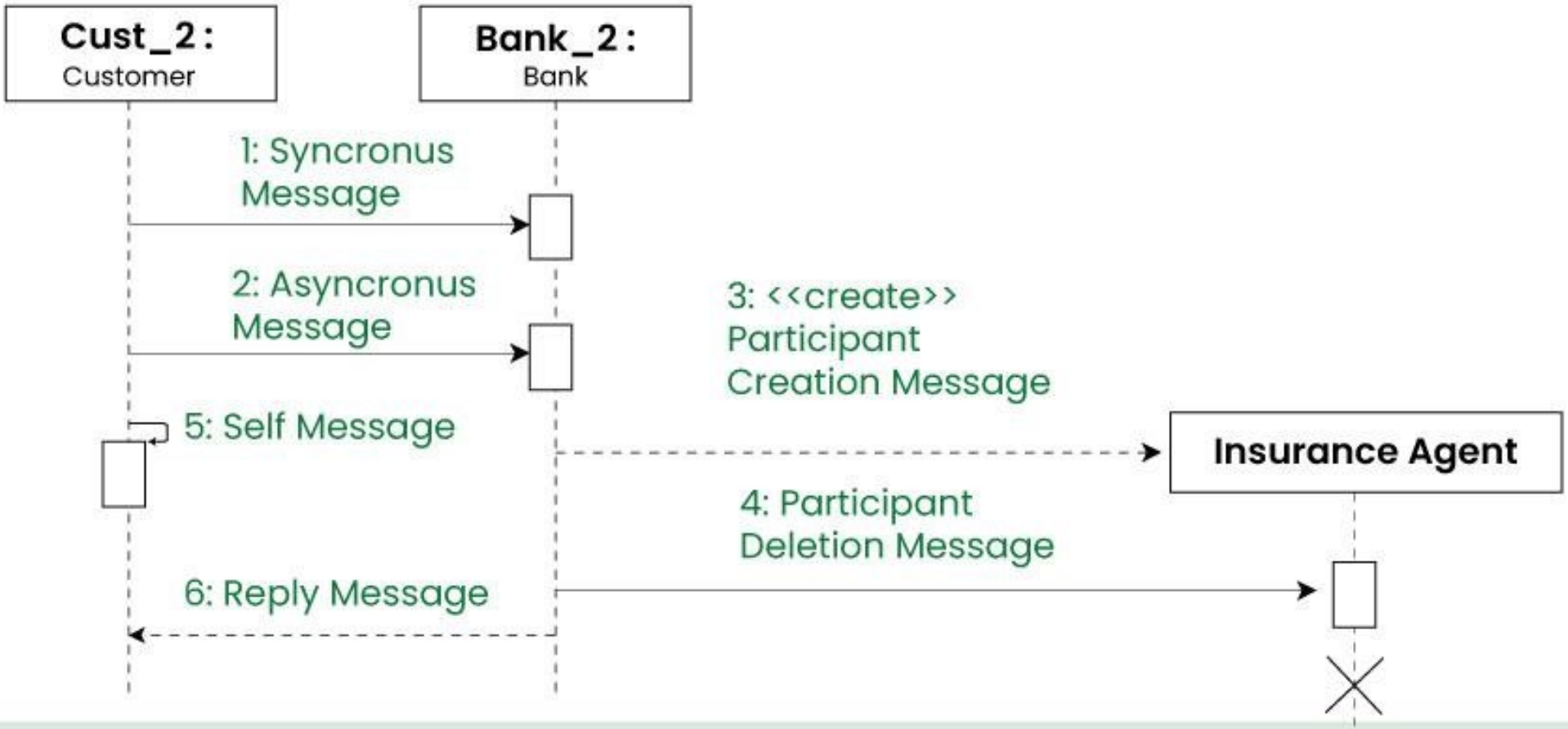



Customer
APP
Auth Server
Bank Server
Login
verify password
Alt
return JWT
[if password is valid]
return OK
[Else]
return Invaild

# Các loại - Message (Thông điệp):

- Synchronous (Đồng bộ): là loại phổ biến nhất.
  - Tuần tự từng bước một, lệnh sau đợi lệnh trước thực thi xong
  - Dễ quản lý luồng logic, kết quả rõ ràng.
  - Mũi tên đặc (->), người gửi đợi phản hồi.
- Asynchronous (Bất đồng bộ):
  - Lệnh tiếp theo có thể chạy trước khi lệnh trước hoàn thành
  - Chương trình không bị "treo", giữ cho giao diện người dùng phản hồi tốt
  - Thường dùng trong các hệ thống đa luồng.
  - Mũi tên hở (-->), người gửi không đợi phản hồi.
- Return (Thông điệp trả về):
  - Mũi tên nét đứt (<--), cho thấy dữ liệu được trả về.
  - Có thể bỏ qua nếu không cần thiết để sơ đồ đơn giản hơn.

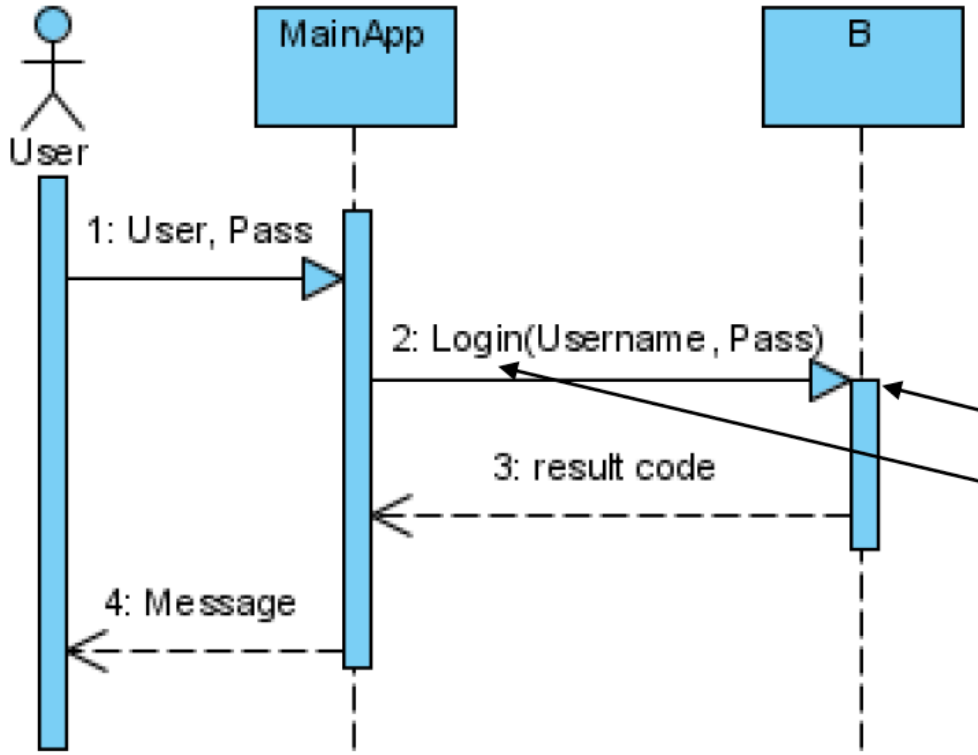
## Different Types of Messages



# Quy trình 6 bước Tạo Sơ đồ Chuỗi

- 1.Thiết lập Ngữ cảnh (Set Context):
  - Xác định sơ đồ này dành cho use case hay kịch bản nào.
  - Vẽ một khung (frame) và đặt tên cho nó.
- 2. Xác định Actor và Object (Identify Actors and Objects):
  - Dựa trên mô hình chức năng và cấu trúc, xác định các actor và đối tượng tham gia.
- 3. Thiết lập Đường sống (Set Lifeline):
  - Vẽ một đường sống (lifeline) nét đứt thẳng đứng bên dưới mỗi actor và object.
- 4. Thêm các Thông điệp (Add Messages):
  - Vẽ các mũi tên từ người gửi đến người nhận theo thứ tự thời gian từ trên xuống dưới.
- 5. Đặt Sự kiện thực thi (Place Execution Occurrence):
  - cho biết khi nào một đối tượng đang hoạt động.
- 6. Xác thực Sơ đồ (Validate):
  - Đảm bảo sơ đồ mô tả đầy đủ tất cả các bước trong kịch bản.

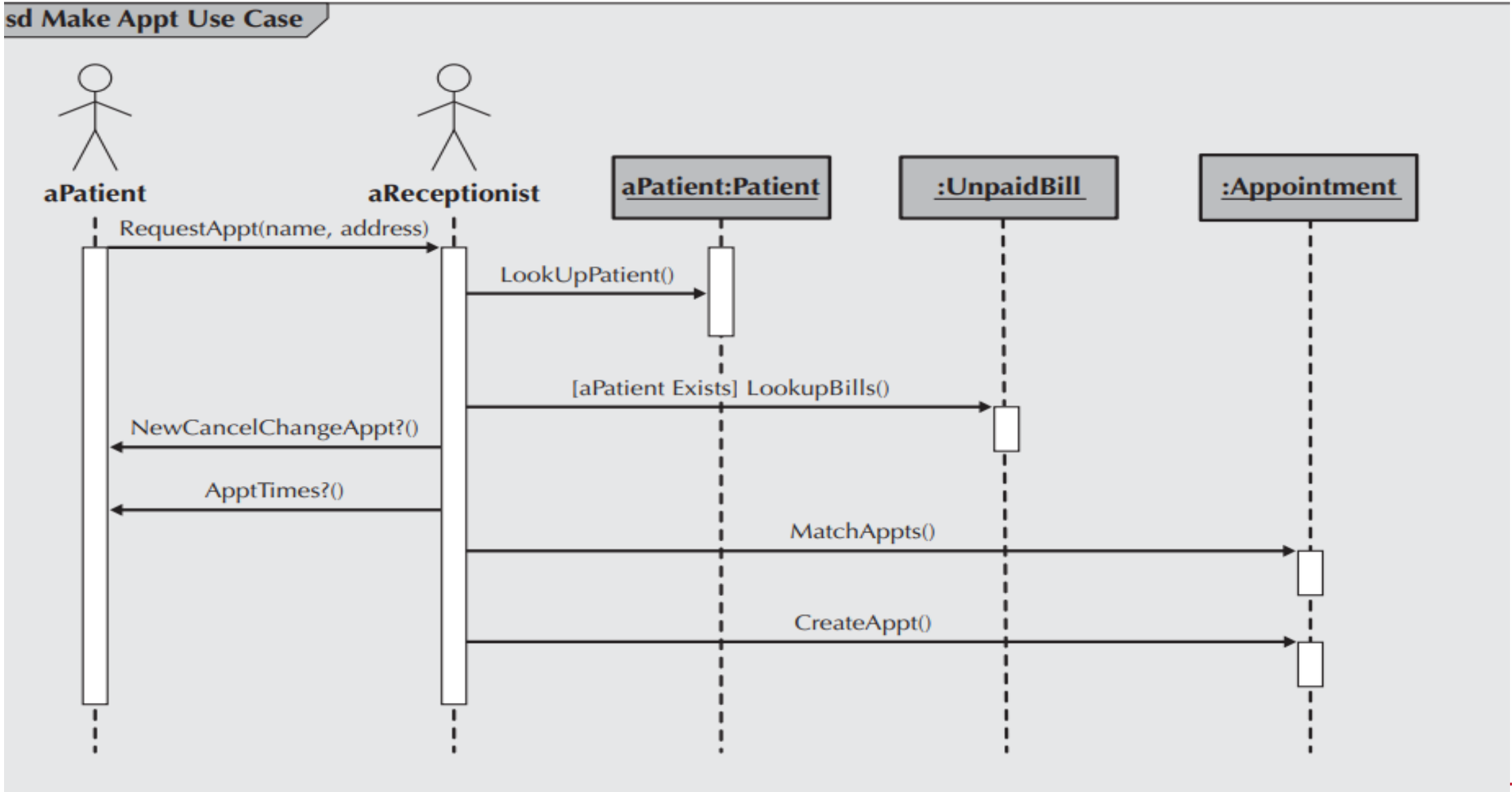
# Ánh xạ biểu đồ tuần tự sang Code




```
Class B { public int Login(string UID, Pass)
{ ..............
}
MainApp
{     .... Nhập User name, password ....
      B objB = new B ();
      bool Result = objB.Login(UID, Password)
      if (Result == true) ....
}
```

# Ví dụ Sơ đồ Chuỗi cho Use Case "Make Old Patient Appt"

- áp dụng những gì đã học vào một ví dụ cụ thể.
  - Chúng ta sẽ phân tích kịch bản một bệnh nhân cũ đặt lịch hẹn mới và xem các đối tượng tương tác với nhau như thế nào."
- Giải thích các bước:
  - 1. aPatient yêu cầu đặt lịch hẹn (RequestAppt).
  - 2. aReceptionist tra cứu thông tin bệnh nhân (LookUpPatient).
  - 3. Nếu bệnh nhân tồn tại [aPatient Exists], aReceptionist tra cứu hóa đơn chưa thanh toán (LookupBills).
  - 4. aReceptionist hỏi bệnh nhân về loại hành động (hẹn mới, hủy, thay đổi).
  - 5. aReceptionist hỏi về thời gian mong muốn.
  - 6. aReceptionist đối chiếu lịch hẹn (MatchAppts).
  - 7. aReceptionist tạo lịch hẹn mới (CreateAppt).


sd Make Appt Use Case
aPatient
aReceptionist
aPatient:Patient
:UnpaidBill
:Appointment
RequestAppt(name, address)
LookUpPatient()
[aPatient Exists] LookupBills()
NewCancelChangeAppt?()
ApptTimes?()
MatchAppts()
CreateAppt()

# interaction overview diagram
# Timing Diagram

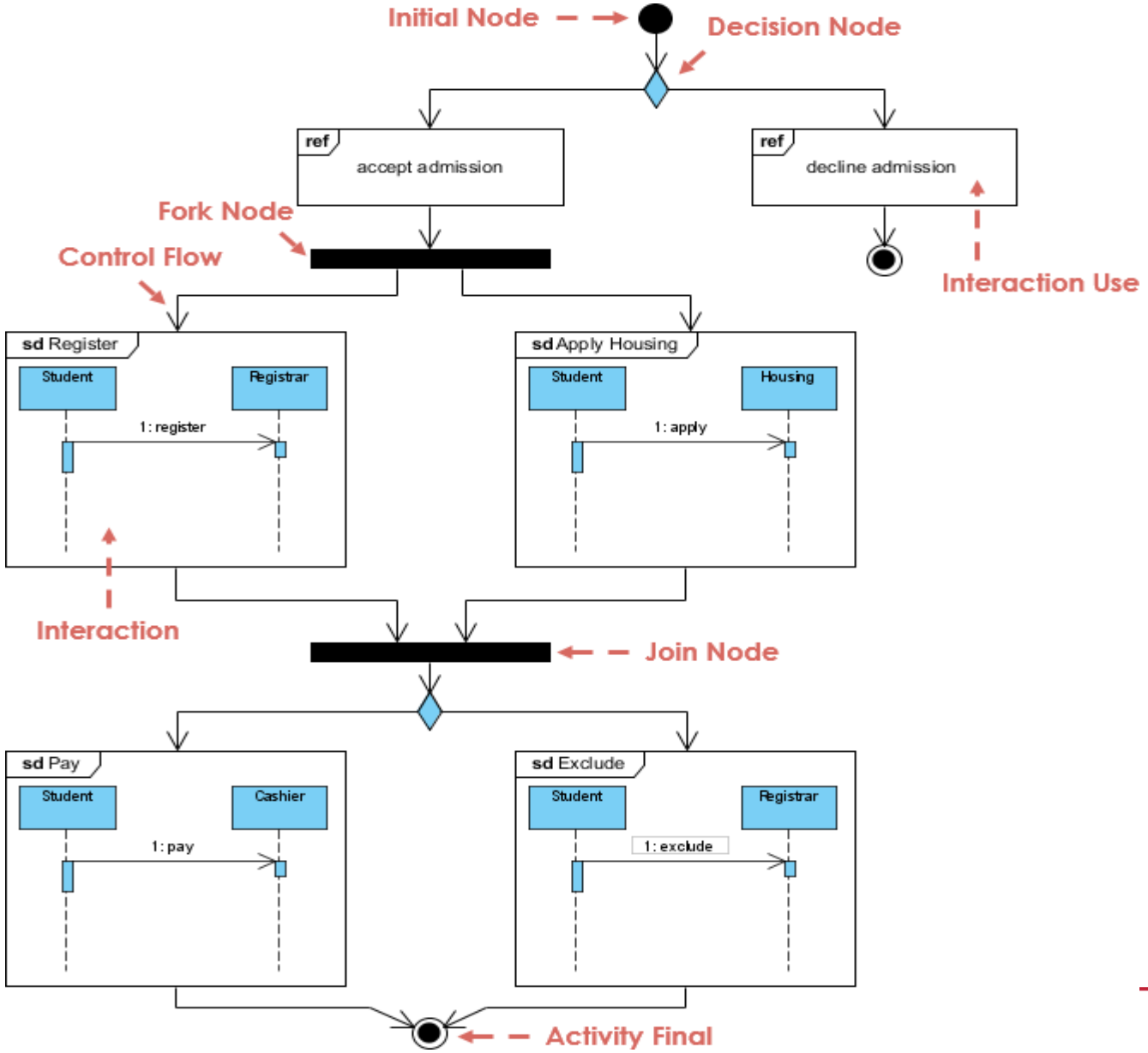
# Sơ đồ Tổng quan Tương

- Interaction Overview Diagram (IOD)
  - Là  sơ đồ hành vi UML cấp cao hoạt động như một "bản đồ của các bản đồ".
  - Nó kết hợp cấu trúc của sơ đồ hoạt động với các chi tiết của sơ đồ tương tác (như sơ đồ trình tự) để thể hiện luồng điều khiển trong một hệ thống phức tạp.
- Thường được sử dụng tác khi
  - cần mô hình hóa một quy trình kinh doanh hoặc luồng công việc của hệ thống liên quan đến nhiều kịch bản hoặc logic ra quyết định phức tạp.
- Nó đặc biệt hữu ích cho các nhà thiết kế hệ thống
  - truyền đạt logic cho các bên liên quan mà không bị sa lầy vào trao đổi thông điệp riêng lẻ.

# Các thành phần chính

- Sử dụng tương tác (ref):
  - Tham chiếu đến một sơ đồ tương tác hiện có (ví dụ: sơ đồ trình tự).
  - Nó được biểu thị bằng một khung hình chữ nhật có chữ "ref" ở góc.
- Interaction -Tương tác (sd):
  - Sơ đồ tương tác nội tuyến được hiển thị trực tiếp trong một khung.
- Các nút điều khiển:
  - Được mượn từ sơ đồ hoạt động, chúng bao gồm các nút Khởi tạo/Kết thúc , Quyết định (hình thoi) và Ngã ba/Kết hợp (hình thanh) để quản lý logic và các luồng song song.
- Luồng điều khiển:
  - Các mũi tên kết nối các phần tử này để thể hiện trình tự thực thi.

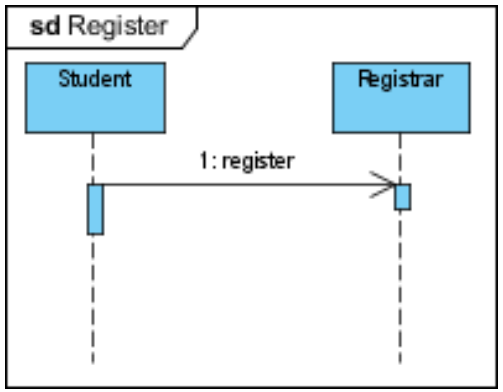
# Interaction Overview Diagram



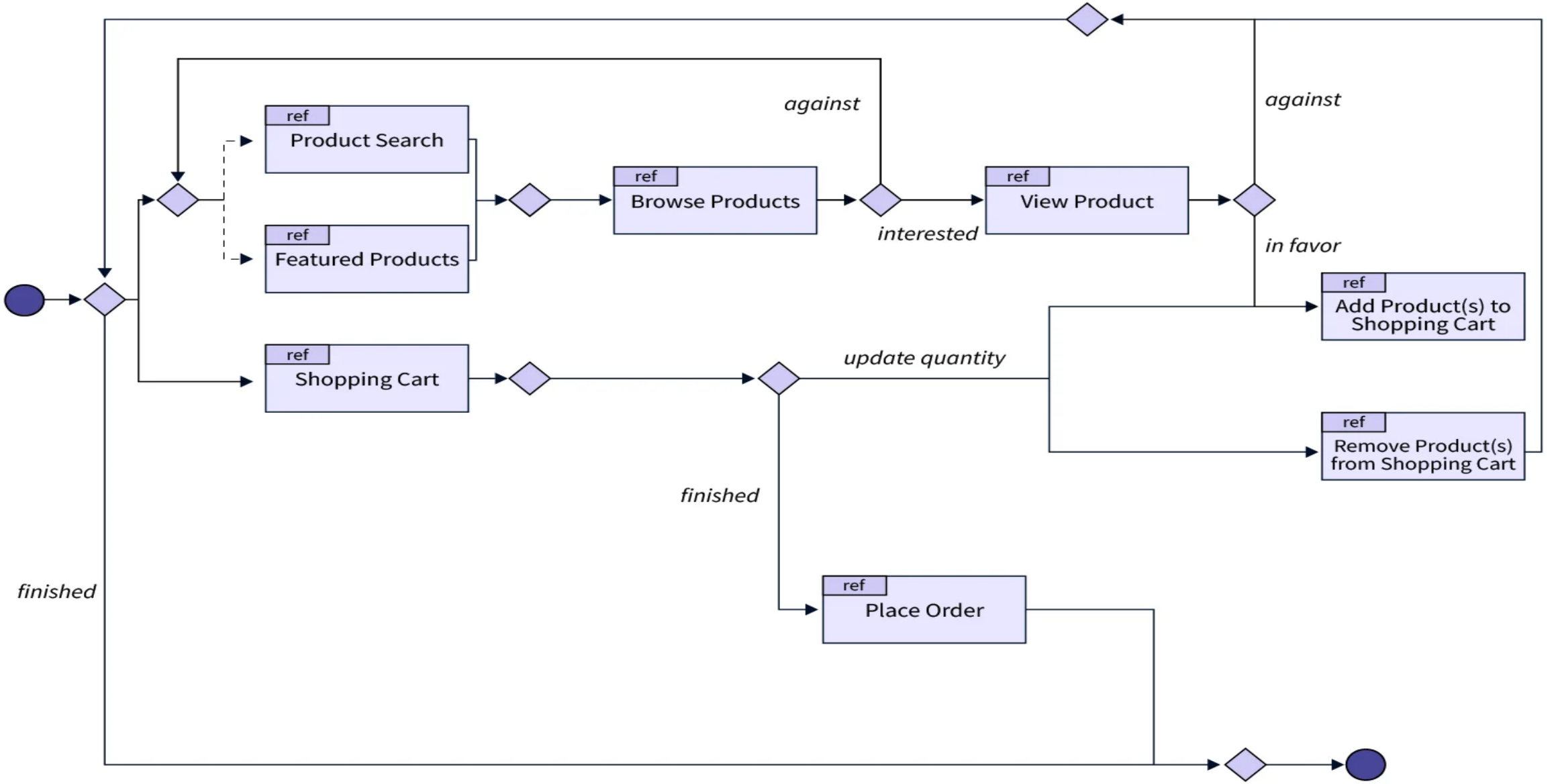

- Ví dụ trên minh họa một sinh viên đã được nhận vào trường đại học.
  1. SV chấp nhận hoặc từ chối nhập học.
  2. sinh viên đăng ký các lớp học và xin chỗ ở.
  3. trả phí cho phòng đăng ký.
     - Nếu không nhận được khoản thanh toán đúng hạn,
     - sinh viên sẽ bị phòng đăng ký đuổi học.

# Mục đích và lợi ích

- Phân tích sự phức tạp:
  - Công cụ này đơn giản hóa các sơ đồ trình tự khổng lồ bằng cách chia chúng thành các tương tác nhỏ hơn, dễ quản lý hơn.
- Trực quan hóa logic:
  - Nó hiển thị rõ ràng các đường dẫn if-then-else và các quy trình song song mà nếu không sẽ rất khó theo dõi trong một sơ đồ tuyến tính duy nhất.
- Điều hướng cấp cao:
  - Cho phép các bên liên quan nhìn thấy "bức tranh tổng thể" về hoạt động của hệ thống, đồng thời cung cấp khả năng đi sâu vào các chi tiết kỹ thuật.

| Loại nút | Ký hiệu |
| --- | --- |
| Sự tương tác<br>• Sơ đồ tương tác thuộc bất kỳ loại nào cũng có thể xuất hiện trực tiếp dưới dạng Lời gọi hoạt động. |  |
| Sử dụng tương tác<br>• Các sơ đồ trình tự lớn và phức tạp có thể được đơn giản hóa bằng cách sử dụng tương tác.<br>• Việc tái sử dụng một số tương tác giữa nhiều tương tác khác cũng khá phổ biến. |  |

# Webstore


ref
Product Search
ref
Featured Products
ref
Browse Products
ref
View Product
ref
Shopping Cart
ref
Add Product(s) to Shopping Cart
ref
Remove Product(s) from Shopping Cart
ref
Place Order
against
against
interested
in favor
update quantity
finished
finished

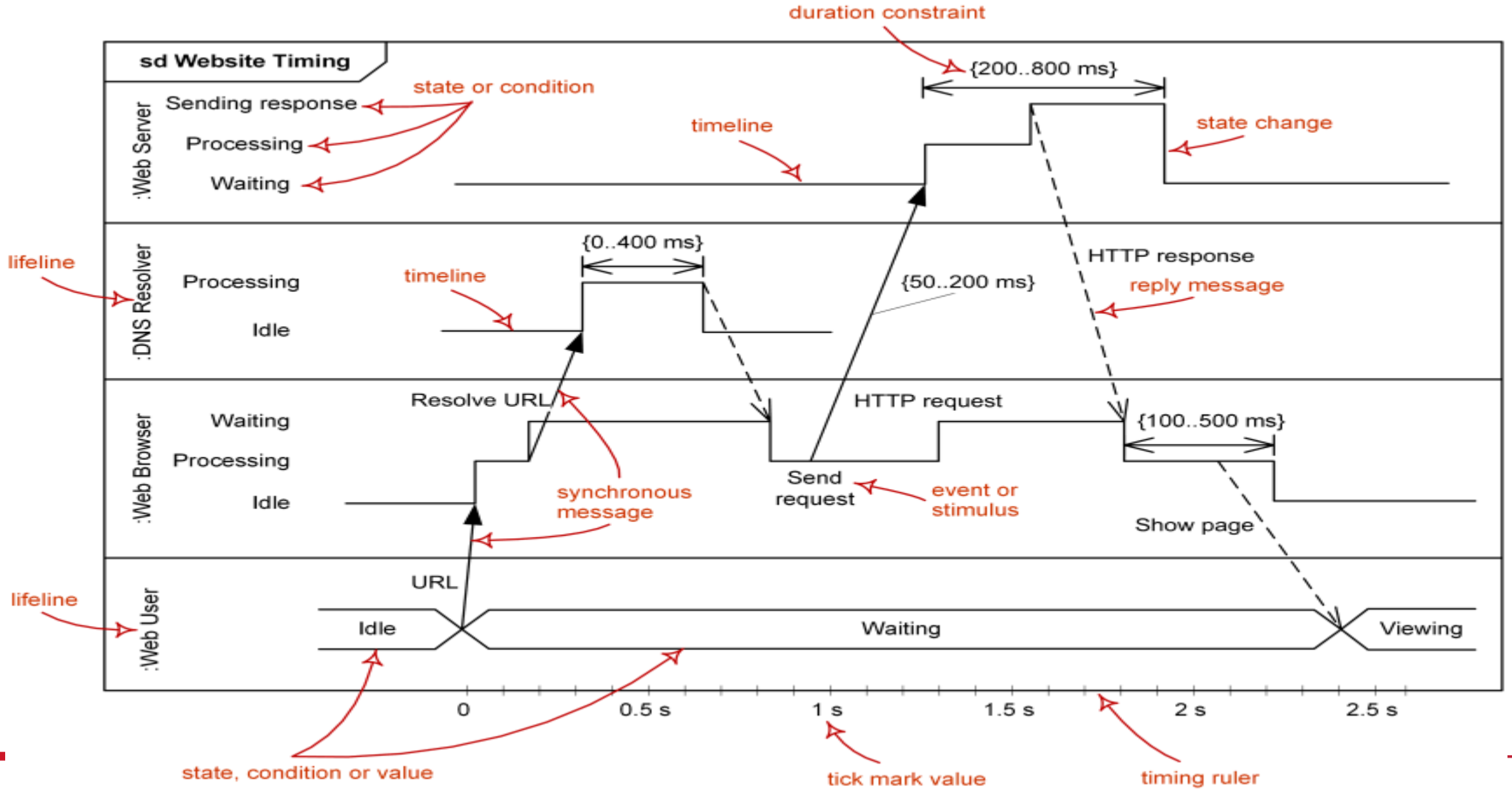
# Timing Diagram in UML

- Timing Diagram
  - là Một loại sơ đồ tương tác chuyên dụng được sử dụng để trực quan hóa hành vi của các đối tượng hoặc thành phần trong một khoảng thời gian cụ thể.
- Khác với sơ đồ trình tự, vốn ưu tiên thứ tự các thông điệp,
  - sơ đồ thời gian tập trung vào thời điểm xảy ra sự thay đổi trạng thái và thời lượng của các trạng thái đó.
- Ứng dụng phổ biến
  - Hệ thống thời gian thực:
    - độ chính xác là yếu tố then chốt,
    - thiết bị y tế hoặc bộ điều khiển đèn giao thông.
  - Phân tích hiệu năng:
    - Xác định các điểm nghẽn và độ trễ trong các quy trình phần mềm.
  - Hệ thống nhúng

# Các thành phần cốt lõi

- Lifeline:
  - ngăn ngang riêng biệt Biểu thị một người tham gia trong tương tác,
- State Timeline:
  - trục biểu thị thời gian, thể hiện sự chuyển đổi của một đối tượng giữa các trạng thái riêng biệt khác nhau.
- Duration Constraint:
  - Ràng buộc về thời gian tối đa người tham gia phải duy trì ở một trạng thái cụ thể (ví dụ: {1..5s}).
- Time Constraint:
  - quy tắc xác định thời điểm một sự kiện hoặc sự thay đổi trạng thái xảy ra.
- Timing Ruler: Một thang khung đo lường cho các khoảng thời gian

# Medical domain - Stages of Alzheimer's Disease



# Tóm lược Mô hình tương tác (Interaction Models):

- Mục đích: Minh họa luồng thông điệp và hành động giữa các thành phần hoặc đối tượng khác nhau trong một hệ thống
- Tập trung: Cách người dùng và thành phần thực hiện mọi việc và giao tiếp với nhau.

- Thuận lợi:
  - Lấy người dùng làm trung tâm: Tuyệt vời để hiểu và thiết kế luồng người dùng trực quan và hành vi sản phẩm.
  - Độ rõ ràng của hệ thống: Cung cấp góc nhìn tổng quan về cách các bộ phận khác nhau của hệ thống hoạt động cùng nhau.
- Nhược điểm:
  - Chi tiết bên trong hạn chế: Không hiển thị hoạt động bên trong hoặc trạng thái của từng vật thể.
  - Độ phức tạp đối với các hệ thống lớn: Có thể trở nên phức tạp và khó quản lý đối với các hệ thống rất lớn hoặc phức tạp.

# CRUDE
# VÀ KIỂM ĐỊNH

# Tại sao Phân tích CRUD ?

- CRUD là ngôn ngữ chung để mô tả cách hệ thống vận hành.
- Nó giúp các nhà phát triển đảm bảo rằng
  - mọi tương tác của người dùng đều dẫn đến một sự thay đổi trạng thái hợp lệ và có kiểm soát
- Nếu bạn có một lớp Product, các thao tác CRUD sẽ là:
  - Create: Nhập sản phẩm mới vào kho.
  - Read: Xem chi tiết thông tin sản phẩm.
  - Update: Thay đổi giá hoặc số lượng tồn kho.
  - Delete: Gỡ bỏ sản phẩm khi ngừng kinh doanh.

# Crud

- 1983 Thuật ngữ Crud - James Martin Managing the Data-base Environment (Quản lý Môi trường Cơ sở Dữ liệu)
- Crud mở rộng sang xác thực
  - logic nghiệp vụ (Business Logic Validation) và
  - ràng buộc trạng thái (State Constraints),
  - đảm bảo hệ thống tuân theo các quy tắc phức tạp
- CRUD và Behavior Model (Mô hình hành vi)
  - giúp kết nối giữa dữ liệu tĩnh và các hoạt động thực tế của hệ thống.
  - biến các sơ đồ tĩnh thành một quy trình nghiệp vụ hoàn chỉnh,
  - đảm bảo mọi thực thể dữ liệu đều được quản lý xuyên suốt vòng đời

# Ma trận CRUD trong Mô hình hành vi

- CRUD
  - Là một ma trận cho thấy sự tương tác giữa các đối tượng.
- Trong UML, CRUD không phải là một biểu đồ,
  - mà là tập hợp các hành vi mà một đối tượng thực hiện
- Ma trận CRUD giúp chúng ta tóm tắt tất cả các loại tương tác giữa các đối tượng trong một ma trận duy nhất."
  - Mỗi tương tác được dán nhãn bằng một chữ cái: Create (Tạo), Read (Đọc), Update (Cập nhật), và Delete (Xóa).
- "Sau khi đã đi sâu vào chi tiết các tương tác và vòng đời của đối tượng,
  - chúng ta cần Ma trận CRUDE như một công cụ để có cái nhìn tổng quan trên toàn hệ thống.

# Validation với CRUD

- Trong phân tích thiết kế,
  - người ta thường lập một ma trận CRUD giữa Lớp (Class) và Hành vi (Behavior) để đảm bảo không bỏ sót chức năng:
- Nếu một lớp dữ liệu tồn tại mà không có hành vi Create,
  - lớp đó "từ trên trời rơi xuống".
- Nếu không có hành vi Read,
  - dữ liệu đó là "dữ liệu chết".
- mô hình hành vi tập trung vào việc tích hợp các quy tắc nghiệp vụ vào chính đối tượng (object),
  - làm cho mỗi thao tác CRUD phải tuân thủ những logic đó trước khi thực sự thay đổi trạng thái của đối tượng.

# CRUD trên Sequence Diagram?

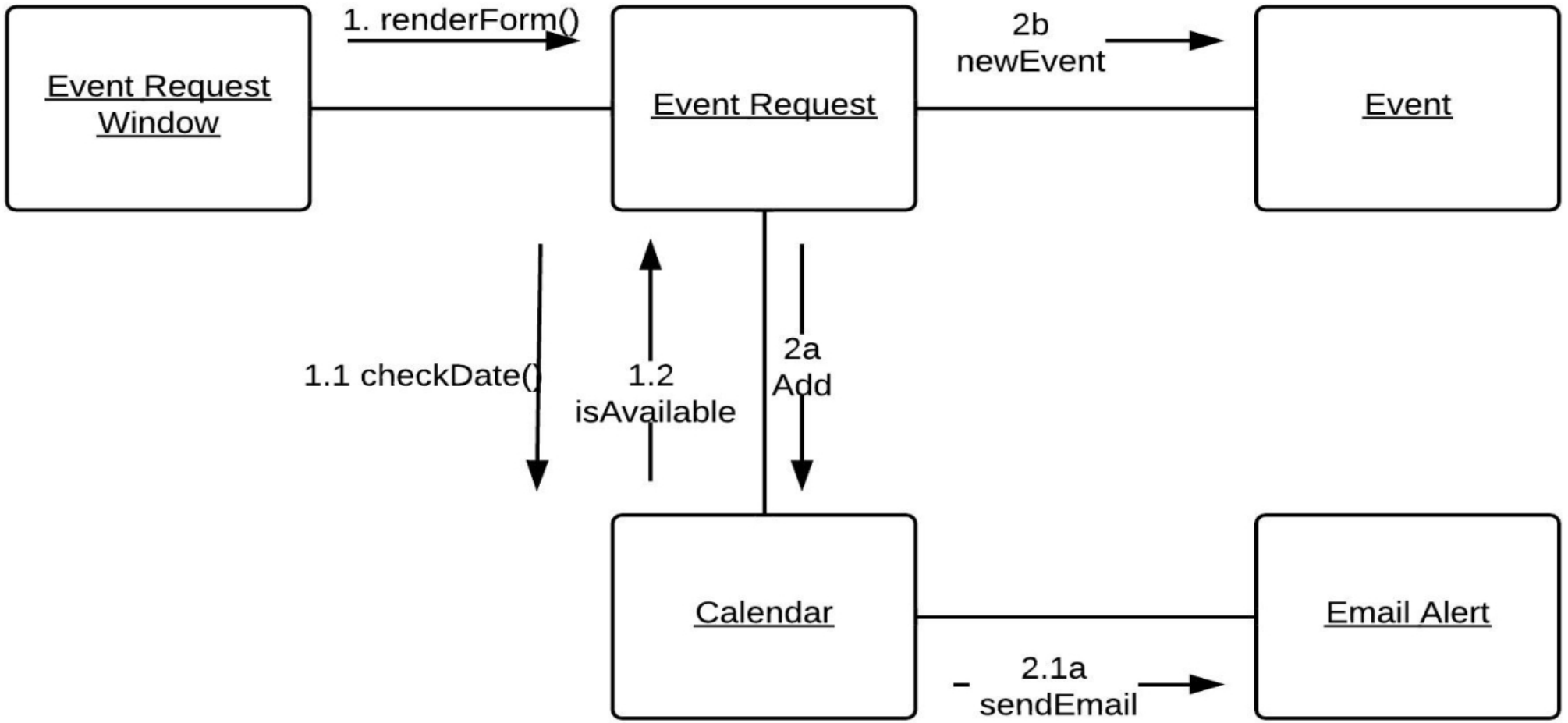
- Trong thiết kế hệ thống, việc kết hợp CRUD vào Sequence Diagram giúp trực quan hóa cách các đối tượng tương tác để thực hiện các thao tác dữ liệu cơ bản.
- Lợi ích
  - Các định phương thức:
    - Giúp lập trình viên biết chính xác cần viết những hàm (method) nào trong Class.
  - Kiểm soát luồng dữ liệu:
    - Đảm bảo không có bước nào bị bỏ sót (ví dụ: quên kiểm tra quyền truy cập trước khi Xóa).
  - Giao tiếp nhóm:
    - Giúp các bên liên quan hiểu rõ cách một tính năng (như "Đăng ký tài khoản" - Create) hoạt động từ giao diện đến cơ sở dữ liệu.

# CRUD và Communication Diagram

- CRUD trong Communication Diagram mô tả cách các thành phần tương tác để thực hiện các chức năng cơ bản của dữ liệu

- Trong Communication Diagram), các thông điệp truyền đi giữa các đối tượng chính là sự cụ thể hóa các thao tác CRUD:
  1. Actor gửi yêu cầu (ví dụ: "Nhấn nút Lưu").
  2. Boundary Object (Giao diện) nhận lệnh và chuyển cho Controller.
  3. Controller thực hiện logic và gọi lệnh Create hoặc Update tới Entity Object (Dữ liệu).
- Communication Diagram sẽ thể hiện trực quan luồng đi của lệnh CRUD đó qua từng lớp của hệ thống.

# CRUD và Communication Diagram



# Cách xây dựng và Ý nghĩa của Ma trận CRUDE

- Cách xây dựng:
  - Tạo một ma trận với các lớp/tác nhân ở cả hàng và cột.
  - Điền vào các ô để chỉ ra loại tương tác.
    - Ví dụ, nếu lớp ở hàng A tạo một thể hiện của lớp ở cột B, điền 'C' vào ô (A, B).
- Ý nghĩa:
  - Xác thực (Validation):
    - Giúp kiểm tra xem mỗi lớp có được tạo (C) và xóa (D) hay không. Một lớp không bao giờ được tạo ra là một dấu hiệu của lỗi trong mô hình.
  - Nhận dạng đối tượng phức tạp:
    - Các lớp có nhiều mục C, U, D trong cột của chúng có thể là các đối tượng phức tạp, là ứng cử viên cho Sơ đồ Máy trạng thái.
  - Hỗ trợ phân cụm lớp:
    - Các lớp tương tác nhiều với nhau có thể được nhóm lại thành các collaboration hoặc partition.

# Ví dụ Ma trận CRUDE cho Use Case "Make Old Patient Appt"

- "Hãy xem xét ma trận CRUDE cho kịch bản đặt lịch hẹn của bệnh nhân cũ để hiểu rõ hơn cách nó hoạt động."
- Giải thích các ô:
  - Ô (Receptionist, Appointment):
    - Tạo (C) một đối tượng Appointment mới khi đặt lịch hẹn
      - (ví dụ: qua thông điệp CreateAppt()).
    - Có thể Đọc (R) thông tin của một lịch hẹn đã có.
    - Có thể Cập nhật (U) một lịch hẹn (khi thay đổi lịch).
    - Có thể Xóa (D) một lịch hẹn (khi hủy lịch).
  - Ô (PatientList, Patient): R - Read
    - Khi Receptionist tìm kiếm, đối tượng PatientList sẽ đọc (R) thông tin từ các đối tượng Patient riêng lẻ để trả về kết quả.
  - Ô (Appointments, Appointment): R - Read
    - đối tượng Appointments (danh sách lịch hẹn) sẽ đọc (R) thông tin từ các đối tượng Appointment cụ thể để xác định các khung giờ trống.

| | Receptionist | PatientList | Patient | UnpaidBills | Appointments | Appointment |
|---|---|---|---|---|---|---|
| Receptionist | | R | U | R | R | C, R, U, D |
| PatientList | | | R | | | |
| Patient | | | | | | |
| UnpaidBills | | | | | | |
| Appointments | | | | | | R |
| Appointment | | | | | | |

# Tầm quan trọng của CRUD trong OOAD

- Xác định phương thức (Methods):
  - Khi thiết kế Class Diagram, mỗi thực thể thường đi kèm với các phương thức CRUD tương ứng
  - (ví dụ: addUser(), getUser(), updateUser(), deleteUser()) .
- Thiết kế Database:
  - CRUD giúp ánh xạ (Mapping) từ các thuộc tính của đối tượng trong code sang các cột trong bảng của cơ sở dữ liệu quan hệ
- Xác định Use Case:
  - CRUD giúp liệt kê đầy đủ các chức năng mà người dùng có thể thực hiện
  - (ví dụ: "Quản lý sản phẩm" thực chất là tập hợp các Use Case CRUD)
- Phân quyền (Authorization):
  - Giúp xác định ai có quyền thực hiện thao tác nào
  - (ví dụ: User chỉ được Read, Admin được Update/Delete)

# Tóm tắt Nội dung

- Mô hình hóa hành vi mô tả cách các đối tượng hợp tác và thay đổi theo thời gian để thực hiện các chức năng của hệ thống.
- Sơ đồ Máy trạng thái Hành vi mô tả vòng đời của một đối tượng phức tạp.
- Sơ đồ tưởng tác
  - Sơ đồ Chuỗi tập trung vào thứ tự thời gian của các thông điệp.
  - Sơ đồ Giao tiếp tập trung vào cấu trúc liên kết giữa các đối tượng.
  - IOD cung cấp cái nhìn tổng quan về cách các tương tác kết nối trong hệ thống
- Ma trận CRUDE cung cấp một cái nhìn tổng quan và là công cụ xác thực mạnh mẽ.
  - Việc xác minh và xác thực là rất quan trọng để đảm bảo các mô hình nhất quán và chính xác.

HUST

hust.edu.vn fb.com/dhbkhn